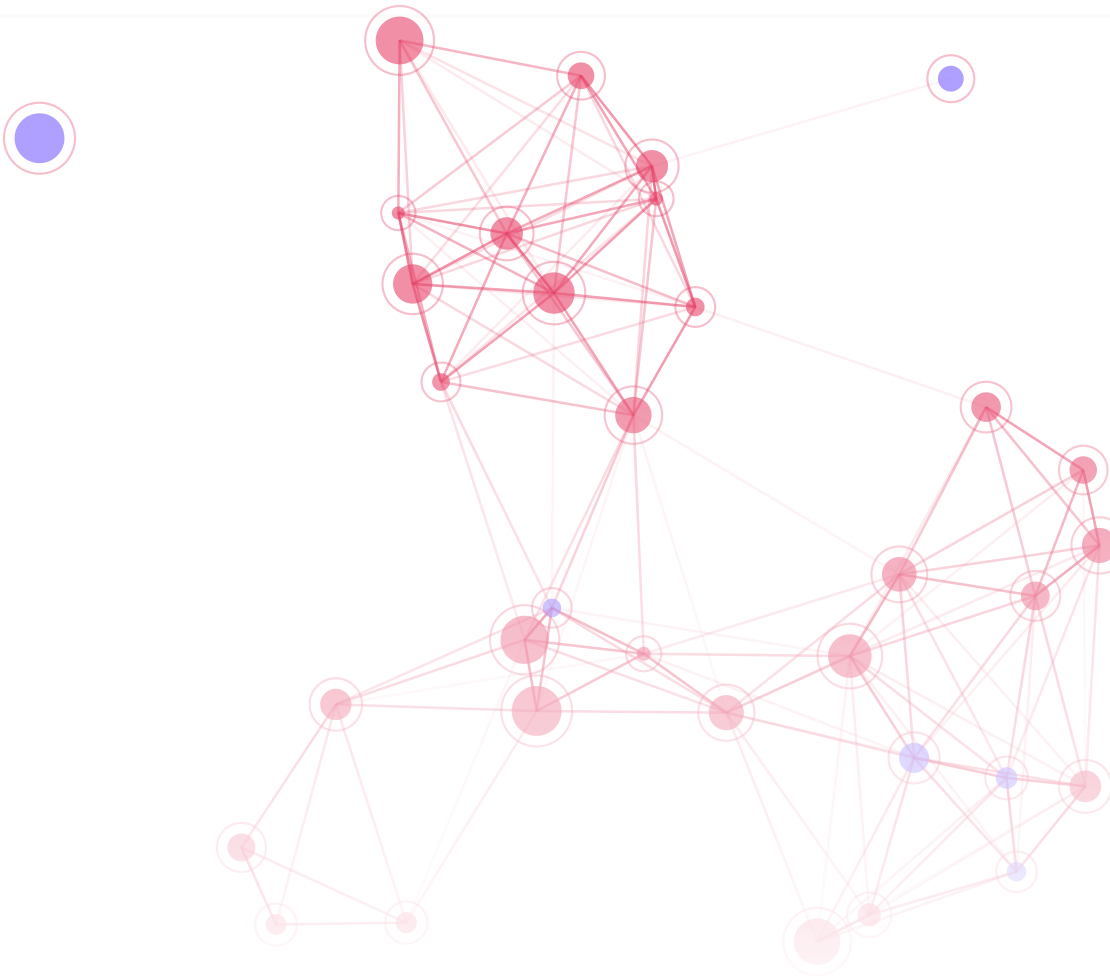


A BEGINNER'S MANUAL

n8n

Automate your work and build AI agents, visually, one node at a time, from your first click to your own server

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

n8n

Automate your work and build AI agents, visually, one node at a time, from your first click to your own server

n8n (say "n-eight-n," short for "nodemation") is a visual automation tool. You connect building blocks, called **nodes**, on a canvas to make **workflows** that run on their own: "every morning, check the weather and email me," "when someone fills in my form, add them to a spreadsheet and send a welcome message," or "when a support email arrives, have an AI draft a reply."

n8n connects to more than a thousand apps, has powerful AI features built in, and, unlike most automation tools, you can run it on your own computer or server for free. This manual assumes no technical background. It takes you from your first click to data, expressions and code, AI agents with memory and knowledge, MCP, a dozen complete step-by-step recipes, and running your own reliable, secure n8n server.

HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

Contents

01 Meet n8n 10

1.1 What is workflow automation? 10
1.2 What can you automate? 11
1.3 How n8n compares 11
1.4 What “fair-code” means for you 12
1.5 Cloud or self-hosted? 12
1.6 How this manual is organised 12

02 Key Ideas in Five Minutes 14

2.1 Items: the most important idea 15
2.2 Nodes come in a few families 15
2.3 Draft versus published 16
2.4 Three kinds of run 16

03 Cloud or Self-Hosted? Making the Choice 17

3.1 The two options in plain English 17
3.2 Side by side 18
3.3 Choose n8n Cloud if… 18
3.4 Choose self-hosting if… 18
3.5 A quick decision guide 19
3.6 The self-hosted editions 19
3.7 Switching later 20

04 Get n8n Running 21

4.1 Option A: n8n Cloud (recommended) 21
4.2 Option B: self-host on your own computer 22
4.3 Create the owner account 24
4.4 Turn on two-factor authentication 25

05 Self-Hosting on a Server 26

5.1 The pieces, explained 27
5.2 Step 1: rent a server 27
5.3 Step 2: point your domain at it 28
5.4 Step 3: install Docker and open the firewall 28

5.5 Step 4: create the project folder and secrets 28
5.6 Step 5: tell Caddy about your domain 29
5.7 Step 6: write the Compose file 30
5.8 What those settings do 31
5.9 Step 7: start everything 32
5.10 Everyday commands 32
5.11 Other ways to self-host 32
5.12 If it doesn't work 33

06 **A Tour of the Editor** 34
6.1 The Overview page 34
6.2 The main areas 35
6.3 Adding nodes 35
6.4 Connecting nodes 35
6.5 The node details view 35
6.6 Fixed values and expressions 36
6.7 Sticky notes and groups 36
6.8 The command bar 36
6.9 Keyboard shortcuts worth learning 36
6.10 The Executions tab 37
6.11 Settings you'll visit 37

07 **Your First Workflow: A Morning Weather Email** 38
7.1 Finding your latitude and longitude 40
7.2 What just happened, node by node 40
7.3 Add a decision 40
7.4 Check that it really ran 41
7.5 Make it yours 41

08 **How Data Flows: Items, JSON and Binary** 42
8.1 JSON in five minutes 42
8.2 The shape of n8n data 43
8.3 How nodes process many items 44
8.4 Inspecting data 44
8.5 Files: binary data 44
8.6 Splitting and combining items 45
8.7 Item linking 46
8.8 Pinning and mocking data 46

09 **Expressions: Little Formulas, Big Power** 47

9.1 Mapping by dragging 47
9.2 Expression mode 47
9.3 Expressions cheat sheet 48
9.4 Reaching your data 48
9.5 Built-in variables 49
9.6 Helper functions 49
9.7 Functions for text, numbers and lists 49
9.8 Dates and times with Luxon 50
9.9 Expressions or nodes? 51
9.10 Worked example: a friendly order summary 51
9.11 Common expression errors 51

10 **The Code Node: JavaScript and Python** 53

10.1 When to use code 53
10.2 The two modes 54
10.3 What goes in and what comes out 54
10.4 Five JavaScript recipes 55
10.5 Built-ins you can use in code 56
10.6 Python 56
10.7 Libraries in JavaScript 57
10.8 Keeping items linked 57
10.9 Let AI write it 57
10.10 Common Code node errors 58

11 **Core Nodes: The Toolkit You'll Use Every Day** 59

11.1 Edit Fields (Set) 59
11.2 Filter 60
11.3 If 60
11.4 Switch 60
11.5 Merge 60
11.6 Loop Over Items (Split in Batches) 61
11.7 HTTP Request 61
11.8 Wait 62
11.9 Everyday helpers 62

12 **Triggers: How Workflows Start** 64

12.1 Schedule Trigger 65

12.2 Webhook 66
12.3 Test URLs versus production URLs 66
12.4 n8n Form Trigger 67
12.5 Chat Trigger 67
12.6 App triggers: instant or polling? 68
12.7 Other triggers 68

13 **Connecting Your Apps: Credentials and OAuth** 69
13.1 Credentials 69
13.2 How OAuth works, in plain English 70
13.3 Managed versus custom OAuth 70
13.4 Walkthrough: Google OAuth on self-hosted n8n 70
13.5 Restricting where a credential can be used 71
13.6 Fixed and end-user credentials 71
13.7 When there's no node: HTTP Request 71
13.8 Sharing credentials 72
13.9 Security habits 72

14 **Sub-workflows: Build Once, Reuse Everywhere** 73
14.1 Why bother? 73
14.2 Create the sub-workflow 73
14.3 Call it from another workflow 74
14.4 How data passes between workflows 75
14.5 Convert part of a workflow 75
14.6 Ideas for your library 75

15 **Data Tables: A Spreadsheet Inside n8n** 76
15.1 Good uses 76
15.2 Create a table 76
15.3 The Data Table node 77
15.4 Worked example: skip what you've seen 77
15.5 Limits 78
15.6 Data tables, variables or a real database? 78

16 **Saving, Publishing and Versions** 79
16.1 Autosave and drafts 79
16.2 Publishing 79
16.3 Workflow history 80
16.4 Exporting and importing 81

16.5 Staying organised 81
16.6 Workflow settings worth knowing 81
16.7 Git and environments (advanced) 82

17 **Handling Errors Gracefully** 83
17.1 Retry On Fail 84
17.2 On Error 84
17.3 Error workflows 84
17.4 Stop and Error 85
17.5 Patterns for common problems 85

18 **Debugging Executions** 86
18.1 The Executions list 86
18.2 Reading a failed execution 87
18.3 Debug in editor 87
18.4 Retrying 87
18.5 Custom execution data 87
18.6 A debugging routine 88
18.7 Decoding common error types 88

19 **AI in n8n: The Basics** 90
19.1 A few AI words 90
19.2 AI steps in a workflow 91
19.3 Connecting a chat model 91
19.4 Getting structured answers 92
19.5 Writing good prompts 92
19.6 Running models locally 93
19.7 Let AI build workflows for you 93

20 **The AI Agent Node in Depth** 94
20.1 How an agent works 94
20.2 The AI Agent node 94
20.3 Writing the system message 96
20.4 Agent options 96
20.5 Memory 96
20.6 Tools 97
20.7 Human review for risky tools 98
20.8 Guardrails 98
20.9 Keeping agents dependable 98

20.10 Agents (first-class, in preview) 98

21 Knowledge for Your AI: Vector Stores and RAG 99

21.1 The idea in plain English 100
21.2 The two halves of RAG 100
21.3 Build a knowledge base 100
21.4 Ask questions with an agent 101
21.5 Choosing a vector store 101
21.6 Keeping it fresh 102
21.7 Improving answers 102

22 MCP: n8n and Other AI Apps 103

22.1 Using MCP servers in your agents 103
22.2 Turn a workflow into an MCP server 104
22.3 Instance-level MCP: let AI build for you 105

23 Recipes: Everyday Automations 106

23.1 Recipe 1: RSS-to-Slack digest 106
23.2 Recipe 2: daily report email 107
23.3 Recipe 3: nightly file backup 108
23.4 Recipe 4: social post scheduler 109

24 Recipes: Business Workflows 110

24.1 Recipe 5: form to CRM 110
24.2 Recipe 6: lead enrichment 111
24.3 Recipe 7: invoice processing with AI 112
24.4 Recipe 8: Notion sync 113
24.5 Recipe 9: web page change monitor 113

25 Recipes: AI Workflows 115

25.1 Recipe 10: email triage with AI 115
25.2 Recipe 11: AI support chatbot with RAG 116
25.3 Recipe 12: Telegram assistant bot 117

26 Running Your Own n8n: Operations 119

26.1 Backups 120
26.2 Updating 121
26.3 Environment variables you'll actually use 122
26.4 Keeping the database small 123

26.5 Scaling with queue mode 123
26.6 Security hardening checklist 123
26.7 Command-line toolbox 124

27 **What n8n Really Costs** 125
27.1 What counts 125
27.2 n8n Cloud plans 126
27.3 Self-hosted costs 126
27.4 AI costs 127
27.5 Ten ways to use fewer executions 127

28 **Templates and Community** 128
28.1 Templates 128
28.2 Community nodes 129
28.3 Getting help 129
28.4 Keep learning 129

29 **Troubleshooting** 130
29.1 Triggers and publishing 130
29.2 Data and expressions 131
29.3 Credentials and APIs 131
29.4 AI 132
29.5 Self-hosting 132

Quick Reference 133

Glossary 134

CHAPTER 1

Meet n8n

Welcome. If you have ever copied information from an email into a spreadsheet, or checked the same website every morning, you already know the kind of work n8n is built to take off your hands. This chapter explains what n8n is, how it differs from similar tools, and what you will be able to build by the end of this manual.

IN THIS CHAPTER

- What workflow automation is, in plain English
- How n8n compares to Zapier and Make
- What “fair-code” and the Community Edition mean
- Cloud versus self-hosted, in one paragraph
- How this manual is organised and how to use it

1.1 What is workflow automation?

A **workflow** is a recipe that a computer follows automatically. It always starts with a **trigger** (*something happens*), then runs **steps** (*do these things*):

TRIGGER	STEPS
Every weekday at 8am	Get today’s calendar → summarize with AI → send to Slack
A new row in Google Sheets	Create an invoice in Stripe → email the customer
A form is submitted	Add the person to your mailing list → notify you on Telegram
An email arrives at support@	AI sorts it → drafts a reply → creates a task

In n8n, every trigger and every step is a **node**: a box on a canvas. You connect the boxes with lines, and data flows along those lines from left to right.

Think of it like a kitchen. The trigger is the order coming in. Each node is a cook at a station: one chops, one fries, one plates. The lines between nodes are the counter where dishes slide from one cook to the next. You design the kitchen once; after that, every order is handled the same way, day and night.

1.2 What can you automate?

Almost anything that happens on a computer and follows a pattern. Some everyday examples that people build in their first month:

- A daily digest of news from your favourite websites
- Saving email attachments to Google Drive
- Posting new blog articles to social media
- Adding form sign-ups to a CRM
- Reminding customers about unpaid invoices
- Backing up files every night
- A chatbot that answers questions about your documents
- Alerts when a web page changes

If you can describe a task as "when *this* happens, do *that*, then *that*," n8n can probably do it.

1.3 How n8n compares

	N8N	ZAPIER	MAKE
Style	Visual canvas with branches and loops	Simple step-by-step lists	Visual canvas
Self-hosting	**Yes, free**	No	No
Pricing basis	Per workflow *execution*, however many steps	Per task (each step counts)	Per operation
Code when you need it	JavaScript or Python nodes	Limited	Limited
AI agents and MCP	Built in	Yes	Yes
Best for	Power, flexibility, and low cost at scale	The quickest simple automations	Visual mid-complexity flows

The pricing row matters more than it looks. Because n8n charges per **execution** (one full run of a workflow) rather than per step, a twenty-step workflow costs the same as a two-step one. Complex automations stay cheap.

> **NOTE**
> n8n is "fair-code": its source code is public and the self-hosted Community Edition is free to use, including for your own business, under n8n's Sustainable Use License. You only pay for n8n Cloud or for enterprise features.

1.4 What "fair-code" means for you

You don't need to read licences to use n8n, but it helps to know the basics:

- **You can** run the free self-hosted version for yourself or inside your own company, with unlimited workflows and executions.
- **You can** read and change the code.
- **You pay** if you want n8n to host it for you (n8n Cloud), or for business features such as single sign-on and Git version control on a self-hosted server.
- If you want to sell n8n itself as a hosted service to others, read the licence first.

1.5 Cloud or self-hosted?

- **n8n Cloud**: n8n runs it for you. Sign up and start in the browser. Easiest, with a free trial. **Recommended for beginners.**
- **Self-hosted**: you run n8n on your own computer or a server. Free and unlimited, but you're responsible for keeping it running, updated, and secure.

You can start on Cloud and move later. Workflows export and import as files. Chapter 3 helps you decide properly.

1.6 How this manual is organised

The manual has five parts. Read the first part in order; after that, dip in wherever you need.

PART	CHAPTERS	WHAT YOU'LL LEARN
Getting started	1–7	Concepts, choosing and installing n8n, the editor, your first workflow
Building skills	8–18	Data, expressions, code, core nodes, triggers, credentials, reuse, errors, debugging
AI	19–22	AI steps, agents, knowledge bases (RAG), MCP
Recipes	23–25	Twelve complete, step-by-step workflows to copy
Running n8n	26–29	Self-hosting operations, costs, community, troubleshooting

> **TIP**
>
> Keep n8n open in another window while you read, and try each example as you go. Automation is learned by doing: ten minutes of clicking teaches more than an hour of reading.

- [ ] I know what a workflow, trigger and node are
- [ ] I know n8n charges per execution, not per step
- [ ] I know there is a free self-hosted edition and a paid Cloud service
- [ ] I have n8n open (or will after Chapter 4) so I can follow along

CHAPTER 2

Key Ideas in Five Minutes

Before you click anything, learn a handful of words. Every n8n screen, error message and forum answer uses them, and once they click, everything else in this manual gets easier.

IN THIS CHAPTER

- The ten words you need to know
- Items: the single most important idea
- Draft versus published workflows
- Manual, partial and production runs

TERM	MEANING
Workflow	A set of connected nodes that does one job
Node	One block: a trigger, an action in an app, or a logic step
Trigger node	The first node, which starts the workflow (a schedule, a webhook, a new email…)
Item	One piece of data moving through the workflow, like one email or one spreadsheet row. Nodes run once per item
JSON	The text format n8n uses to show data: `{ "name": "Sam", "city": "Austin" }`
Expression	A small formula inside `{{ }}` that pulls in data from earlier nodes
Credential	Saved sign-in details that let a node use an app on your behalf
Execution	One run of a workflow; n8n keeps a log of each
Publish	Make a workflow live, so its triggers fire automatically
Instance	Your own copy of n8n, on Cloud or on your server, with its own address

2.1 Items: the most important idea

When a node outputs **five rows** from a spreadsheet, that's **five items**, and the next node normally runs **five times**, once for each. That's how a "send an email" node can send one personalized email per row without any loops.

Picture a conveyor belt. Each item is a parcel on the belt. Each node is a machine the belt passes through. The machine does its job to every parcel in turn, then passes them all along. You rarely have to think about "for each row, do this": n8n does it for you.

SEE ITEMS IN ACTION

1. **Imagine a sheet with three customers.**
   A Google Sheets node reads it and outputs three items: Ana, Ben and Chloé.
2. **Add a Gmail node after it.**
   Set **To** to the expression `{{ $json.email }}`.
3. **Run it.**
   The Gmail node runs three times and sends three emails, each to the right person, because `$json` always means "the current item."

> TIP
> When something sends one email too many, or one too few, count the items. Nine times out of ten the surprise is that a node produced more (or fewer) items than you expected. Chapter 8 shows how to inspect them.

2.2 Nodes come in a few families

FAMILY	WHAT IT DOES	EXAMPLES
Trigger	Starts the workflow	Schedule, Webhook, Gmail Trigger
App (action)	Does something in another service	Slack, Google Sheets, Notion
Core	Logic and data shaping	If, Merge, Edit Fields, Code
AI	Language models and agents	AI Agent, chat models, vector stores

Some AI nodes are **cluster nodes**: a main **root node** (such as AI Agent) with smaller **sub-nodes** plugged into its underside (a chat model, a memory, some tools). You'll meet them in Chapter 19.

2.3 Draft versus published

n8n saves your edits automatically as a **draft**. Your triggers only run automatically after you click **Publish** (or press Shift + P). Later edits stay in draft until you publish again, so you can safely tinker with a live workflow.

2.4 Three kinds of run

KIND	STARTED BY	COUNTS AGAINST CLOUD QUOTA?
Manual	You, clicking **Execute workflow** or **Execute step**	No
Partial	You, running one node and the ones it depends on	No
Production	A trigger on a published workflow	Yes

Only production runs count as executions on n8n Cloud plans, so test as much as you like.

- [ ] I can explain the difference between a node and an item
- [ ] I know `$json` means “the current item”
- [ ] I know edits stay in draft until I publish
- [ ] I know manual test runs don’t use up my execution allowance

CHAPTER 3

Cloud or Self-Hosted? Making the Choice

There are two ways to get n8n, and both run the same product. The question is simply who looks after the computer it runs on: n8n, or you. This chapter lays out the trade-offs so you can choose with confidence, and reassures you that the choice isn't permanent.

IN THIS CHAPTER

- What each option means in practice
- A side-by-side comparison
- A quick decision guide
- The free self-hosted editions
- How to switch later

3.1 The two options in plain English

n8n Cloud is like renting a fully serviced flat. You sign up, get an address such as `yourname.app.n8n.cloud`, and start building in your browser. n8n handles the server, security patches, updates, backups of the platform and scaling. You pay a monthly fee based on how many workflow runs you need.

Self-hosted is like owning a house. You install n8n on your own computer or on a rented server in a data centre (a **VPS**, or *virtual private server*). The software is free and has no execution limits, but you fix the roof: installation, updates, backups, security and keeping it switched on are your job.

3.2 Side by side

	N8N CLOUD	SELF-HOSTED
Setup time	Minutes	An hour or more
Technical skill	None	Comfortable with a terminal
Software cost	From €20/month (yearly)	Free Community Edition
Server cost	Included	A small VPS, a few euros a month
Execution limits	By plan	None (only your server's power)
Updates and security	Done for you	Your job
Your data's location	n8n's servers	Wherever you choose
Custom code libraries	Limited	Allowed
AI without API keys	Gateway credits	Bring your own keys

3.3 Choose n8n Cloud if...

- you are new to n8n, servers or the command line;
- you want to be building within five minutes;
- you'd rather spend your time on workflows than on maintenance;
- you need webhooks reachable from the internet straight away, with HTTPS already set up.

3.4 Choose self-hosting if...

- you are comfortable in a terminal, or keen to learn;
- you want unlimited executions for a fixed, low server cost;
- your data must stay on your own infrastructure (for example for privacy rules);
- you need custom npm or Python packages in Code nodes, your own custom nodes, or bash commands.

> ☆ **IMPORTANT**
>
> n8n itself recommends self-hosting for people with server experience. Mistakes can lead to data loss, security problems or downtime. If that sounds daunting, start on Cloud. Nothing you learn is wasted: the editor is identical.

3.5 A quick decision guide

DECIDE IN FOUR QUESTIONS

1. **Do you just want to learn?**
   Start a Cloud free trial, or run n8n on your laptop with Docker (Chapter 4). Both are fine for practice.
2. **Will it run real work for you or your business?**
   Then it must be always on. Choose Cloud, or a VPS with Docker Compose (Chapter 5).
3. **Will you run thousands of executions a month?**
   Self-hosting is usually far cheaper at high volume.
4. **Does anyone need to approve where data lives?**
   If yes, self-host somewhere they approve, or ask n8n about Enterprise options.

3.6 The self-hosted editions

Self-hosted n8n comes in four flavours. All run the same software; a licence key unlocks extra features.

EDITION	COST	WHAT YOU GET
Community	Free	Almost the complete feature set
Registered Community	Free	Adds folders, debug in editor and custom execution data
Business	Paid	SSO, environments, projects, Git version control
Enterprise	Paid	Advanced security, scaling and governance

To register a Community instance for the free extras, open `Settings` › `Usage and plan` and select **Unlock**. You'll receive a licence key by email.

> **NOTE**
> Features that need a paid plan include custom variables, environments, external secrets, log streaming, multi-main mode, projects, SSO, workflow and credential sharing, and Git version control. Feature lists change, so check n8n.io/pricing for the current details.

3.7 Switching later

You aren't locked in. Workflows are stored as JSON files that you can download from one instance and import into another (Chapter 16). Credentials (your passwords and tokens) don't travel with downloaded workflows, so you'd reconnect your apps after a move. That takes minutes per app.

- [ ] I know who maintains the server in each option
- [ ] I have picked Cloud or self-hosted for now
- [ ] I know I can move later by exporting workflows
- [ ] If self-hosting for real, I'm ready to read Chapters 5 and 26

CHAPTER 4

Get n8n Running

Time to get your own n8n. This chapter covers the two quickest routes: an n8n Cloud trial in your browser, or n8n on your own computer with Docker. If you want an always-on server with your own domain name, read this chapter first and then Chapter 5.

IN THIS CHAPTER

- Starting an n8n Cloud trial, and what happens when it ends
- The Cloud plans and their limits
- Running n8n on your computer: the one-line setup, Docker, and npx
- Creating the owner account and your first look around

4.1 Option A: n8n Cloud (recommended)

START ON N8N CLOUD

1. **Go to n8n.io** and click **Get started**. Create an account; a free trial starts automatically. No credit card is needed.
2. **Choose a name** for your workspace. It becomes your address, like `yourname.app.n8n.cloud`.
3. **Open the editor.**
   You'll see the Overview page with your workflows list.
4. **Check your time zone.**
   Schedules use it. On Cloud, n8n tries to detect your time zone at sign-up. To change it, open your Cloud dashboard, select **Manage**, and change the **Timezone** dropdown.

What the trial includes

The free trial lasts **14 days** and gives you Pro plan features with the computing power of the Starter plan. Its limits are **1,000 executions**, 5 concurrent executions and one team project. You can upgrade at any time with the **Upgrade** button in the top-right corner, choosing yearly or monthly billing.

> ⚠ **WHEN THE TRIAL ENDS**
>
> If you don't upgrade, the trial expires on its own and no charges occur, but n8n then deletes your workspace. You have 90 days to download your workflows afterwards. Export anything you want to keep before day 14.

The Cloud plans

CLOUD PLAN	PRICE (BILLED YEARLY)	EXECUTIONS PER MONTH	HIGHLIGHTS
Starter	€20/month	2,500	Unlimited workflows, AI agents and MCP, 5 concurrent runs
Pro	€50/month	10,000	More concurrency, admin roles, workflow history, insights
Enterprise	Contact sales	Custom	SSO, advanced security, support with an SLA

Prices from n8n.io/pricing, October 2026. Billed monthly, Starter is €24 and Pro is €60 a month. Every plan includes unlimited users, unlimited workflows and every integration. An **execution** is one full run of a workflow, however many nodes it has. Chapter 27 covers costs in depth, including the self-hosted Business plan.

4.2 Option B: self-host on your own computer

Running n8n on your laptop is free and great for learning. You need **Docker**, a free program that runs software in sealed boxes called **containers**, so n8n can't interfere with the rest of your computer. Install **Docker Desktop** from docker.com, open it once, and leave it running in the background.

The one-line setup (easiest)

On macOS, Linux, or Windows with WSL (Windows Subsystem for Linux), open a terminal and run:

TERMINAL

```
curl -fsSL https://get.n8n.io | sh
```

The script checks Docker is running, creates an `n8n` folder in the current directory, writes the configuration files, downloads n8n and starts it. When it finishes it prints `n8n is running at: http://localhost:5678`.

TO DO THIS	RUN THIS
Stop n8n	`docker compose -f ./n8n/compose.yml down`
Start it again	`docker compose -f ./n8n/compose.yml up -d`
Upgrade	`curl -fsSL https://get.n8n.io | sh -s -- --upgrade`

> **TIP**
> Prefer not to pipe a script from the internet straight into your shell? Download it with `curl -fsSL https://get.n8n.io -o get-n8n.sh`, read it with `less get-n8n.sh`, then run `sh get-n8n.sh`.

> **DANGER**
> The uninstall command, `docker compose -f ./n8n/compose.yml down -v`, deletes **all** your n8n data, including workflows and credentials. The `-v` means "remove volumes." Leave it off when you only want to stop n8n.

With plain Docker

If you'd rather type the Docker command yourself (any system with Docker Desktop installed):

TERMINAL

```
docker volume create n8n_data

docker run -it --rm --name n8n -p 5678:5678 \
  -e GENERIC_TIMEZONE="America/New_York" -e TZ="America/New_York" \
  -v n8n_data:/home/node/.n8n \
  n8nio/n8n
```

What each part means:

PART	MEANING
`docker volume create n8n_data`	Make a storage box that survives restarts
`-p 5678:5678`	Open n8n on port 5678 of your computer
`-e GENERIC_TIMEZONE=…`	Set the time zone schedules use
`-v n8n_data:/home/node/.n8n`	Keep n8n's data in that storage box
`--rm`	Delete the container (not the data) when it stops

Just trying it? If you have Node.js (version 20.19–24) installed, run `npx n8n`.

Then open **http://localhost:5678** in your browser and create the owner account.

> **NOTE**
> The npm installation method is deprecated from n8n 3.0, when n8n will be distributed only through Docker. Use Docker or Docker Compose for anything long-lived.

> **WARNING**
> A self-hosted n8n on your laptop only runs while your laptop is on and awake, and outside services can't reach its webhooks. For always-on automations, use n8n Cloud or a small cloud server (see Chapter 5).

> **TIP**
> Replace `America/New_York` with your own time zone (for example `Europe/London` or `America/Los_Angeles`) so schedules fire at the right local time. Self-hosted n8n uses New York time if you don't set one.

4.3 Create the owner account

The first person to open a new self-hosted instance becomes its **owner**: the administrator who can invite other users and change settings.

SET UP YOUR INSTANCE

1. **Open http://localhost:5678.**
   You'll see the **Set up owner account** screen.
2. **Enter your email, name and a strong password.**
   Store the password in a password manager.
3. **Answer the short survey** (or skip it). It only tailors suggestions.
4. **Register for free extras (optional).**
   Open `Settings` › `Usage and plan` and select **Unlock**. Registering your email unlocks folders, debug in editor and custom execution data, at no cost.

4.4 Turn on two-factor authentication

Two-factor authentication (2FA) means signing in needs your password *and* a code from an app on your phone. Do this on Cloud and self-hosted alike: your n8n holds the keys to all your other apps.

ENABLE 2FA

1. **Open your personal settings.**
   Go to `Settings` › `Personal`.
2. **Select Enable 2FA.**
   n8n shows a QR code. Scan it with an authenticator app such as Google Authenticator, 1Password or Authy.
3. **Enter the six-digit code** from the app in **Code from authenticator app**, then select **Continue**.
4. **Save the recovery codes** somewhere safe, away from your computer. They get you back in if you lose your phone.

- [ ] I have a Cloud trial or a local n8n running
- [ ] I can open the editor in my browser
- [ ] My time zone is set correctly
- [ ] I've created the owner account and turned on 2FA
- [ ] I know `down -v` deletes my data, and plain `down` doesn't

CHAPTER 5

Self-Hosting on a Server

A laptop is fine for learning, but real automations need a computer that never sleeps and that the internet can reach. This chapter walks you through renting a small cloud server and running n8n on it with Docker Compose, a proper PostgreSQL database, your own domain name and automatic HTTPS. Take it slowly: every command is explained.

IN THIS CHAPTER

- What a VPS, a domain, DNS, HTTPS and a reverse proxy are
- Renting and securing a small Linux server
- A complete Docker Compose setup: n8n, PostgreSQL and Caddy
- The environment variables that make webhooks work
- Other routes: the n8n Caddy repository, Traefik, and npm

☆ IMPORTANT

Self-hosting means you are the system administrator. You'll be responsible for updates, backups and security (Chapter 26). If any of this feels like too much, n8n Cloud does all of it for you.

5.1 The pieces, explained

PIECE	WHAT IT IS	ANALOGY
VPS	A rented computer in a data centre, always on	A flat you rent
Domain	A name like `example.com` you buy from a registrar	Your street address
DNS record	A setting that points `n8n.example.com` at your server's IP address	The signpost to your door
HTTPS	Encrypted web traffic, shown by the padlock in the browser	Sealed envelopes instead of postcards
Reverse proxy	A program that receives web traffic and hands it to n8n, handling HTTPS certificates	A receptionist at the front desk
Docker Compose	A tool that starts several containers together from one file	A recipe card for the whole kitchen
PostgreSQL	A robust database for n8n's workflows and history	A filing cabinet built for heavy use

We'll use **Caddy** as the reverse proxy because it fetches and renews free HTTPS certificates from Let's Encrypt on its own, with a three-line configuration file.

5.2 Step 1: rent a server

Any provider works: Hetzner, DigitalOcean, AWS, Azure, Google Cloud and many others. n8n publishes guides for several of them. Choose:

- **Operating system:** Ubuntu LTS (the long-term-support version) is the most common choice and matches the commands below.
- **Size:** n8n's own Hetzner guide says the small CPX11 type is enough for most usage. Choose at least **4 GB RAM and 2 vCPUs** if you plan to turn on n8n Assistant, which runs a code sandbox.
- **Login:** choose an **SSH key** rather than a password. SSH is how you'll log in to type commands; keys are far safer.

When the server is ready, note its **IP address** (four numbers like `203.0.113.10`) and log in from your terminal:

TERMINAL

```
ssh root@203.0.113.10
```

5.3 Step 2: point your domain at it

In your domain registrar's DNS settings, add an **A record**:

RECORD TYPE	NAME	DESTINATION
A	`n8n`	Your server's IP address

This makes `n8n.example.com` lead to your server. DNS changes can take from a few minutes to a few hours to spread. Check with `ping n8n.example.com` from your own computer: when it shows your server's IP, you're ready.

5.4 Step 3: install Docker and open the firewall

On the server, install Docker with Docker's official convenience script, then check it worked:

TERMINAL

```
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

Now set up the firewall. **ufw** ("uncomplicated firewall") comes with Ubuntu. Allow SSH first so you don't lock yourself out, then web traffic:

TERMINAL

```
sudo ufw allow OpenSSH
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
```

Port 80 is plain web traffic (Caddy needs it to prove you own the domain and to redirect visitors to HTTPS). Port 443 is HTTPS. You do **not** open port 5678: only Caddy talks to n8n directly.

5.5 Step 4: create the project folder and secrets

TERMINAL

```
mkdir ~/n8n-compose && cd ~/n8n-compose
openssl rand -hex 32
```

The second command prints a long random string. Copy it: it becomes your **encryption key**, which n8n uses to scramble your saved credentials. Run `openssl rand -hex 32` again for a database password. Now create a file called `.env` with `nano .env` and paste this, replacing

every value:

.ENV

```
N8N_VERSION=2.43.6
N8N_DOMAIN=n8n.example.com
GENERIC_TIMEZONE=Europe/London

N8N_ENCRYPTION_KEY=paste-your-first-random-string-here

POSTGRES_USER=n8n_admin
POSTGRES_PASSWORD=paste-your-second-random-string-here
POSTGRES_DB=n8n
```

Save with Ctrl + O, Enter, then exit with Ctrl + X.

> **TIP**
> `N8N_VERSION` pins the exact n8n version, so an update only happens when you choose. Find the latest stable version on n8n's GitHub releases page (github.com/n8n-io/n8n/releases). At the time of writing the stable release was 2.43.6.

> **DANGER**
> Store a copy of `.env`, especially `N8N_ENCRYPTION_KEY`, in your password manager. If you lose the key, every saved credential in n8n becomes unreadable and must be re-entered.

5.6 Step 5: tell Caddy about your domain

Create a file named `Caddyfile` (no extension) with `nano Caddyfile`:

CADDYFILE

```
n8n.example.com {
    reverse_proxy n8n:5678 {
        flush_interval -1
    }
}
```

Replace `n8n.example.com` with your domain. `reverse_proxy n8n:5678` sends visitors to the container named `n8n` on port 5678. `flush_interval -1` passes streamed data (like live AI chat replies) straight through instead of buffering it.

5.7 Step 6: write the Compose file

Create `compose.yml` with `nano compose.yml`. The file is long, so it's shown in two parts. Paste **part 1** first, then **part 2** directly underneath it in the same file. Indentation (the spaces at the start of lines) matters in YAML, so keep it exactly as shown.

Part 1 defines Caddy and the PostgreSQL database:

COMPOSE.YML (PART 1)

```yaml
services:
  caddy:
    image: caddy:latest
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - caddy_data:/data
      - ./Caddyfile:/etc/caddy/Caddyfile

  postgres:
    image: postgres:18
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
      PGDATA: /var/lib/postgresql/data
    volumes:
      - db_storage:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
```

The `${...}` placeholders are filled in from your `.env` file. The **healthcheck** lets Docker know when the database is ready, so n8n doesn't start too early. Keep the `PGDATA` line: PostgreSQL 18 changed its default data folder, and this line keeps the data where the volume expects it.

Part 2 defines n8n itself and the storage volumes:

COMPOSE.YML (PART 2)

```
  n8n:
    image: docker.n8n.io/n8nio/n8n:${N8N_VERSION}
    restart: always
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: postgres
      DB_POSTGRESDB_PORT: "5432"
      DB_POSTGRESDB_DATABASE: ${POSTGRES_DB}
      DB_POSTGRESDB_USER: ${POSTGRES_USER}
      DB_POSTGRESDB_PASSWORD: ${POSTGRES_PASSWORD}
      N8N_HOST: ${N8N_DOMAIN}
      N8N_PORT: "5678"
      N8N_PROTOCOL: https
      N8N_WEBHOOK_URL: https://${N8N_DOMAIN}/
      N8N_PROXY_HOPS: "1"
      N8N_ENCRYPTION_KEY: ${N8N_ENCRYPTION_KEY}
      NODE_ENV: production
      GENERIC_TIMEZONE: ${GENERIC_TIMEZONE}
      TZ: ${GENERIC_TIMEZONE}
    volumes:
      - n8n_data:/home/node/.n8n
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  caddy_data:
  db_storage:
  n8n_data:
```

5.8 What those settings do

SETTING	WHY IT MATTERS
`DB_TYPE` and `DB_POSTGRESDB_*`	Use PostgreSQL instead of the built-in SQLite file
`N8N_HOST`, `N8N_PROTOCOL`	Tell n8n its public name and that it's served over HTTPS
`N8N_WEBHOOK_URL`	The public address n8n shows and registers for webhooks
`N8N_PROXY_HOPS`	n8n sits behind one proxy (Caddy), so trust its forwarded headers
`N8N_ENCRYPTION_KEY`	Your own key for encrypting credentials
`TZ`, `GENERIC_TIMEZONE`	The time zone for schedules and dates
`restart: always`	Start again automatically after a crash or reboot

> **NOTE**
> `N8N_WEBHOOK_URL` is available from n8n 2.30.0. The older name, `WEBHOOK_URL`, still works but is deprecated from 2.35.0, so you'll see it in older tutorials.

5.9 Step 7: start everything

TERMINAL

```
docker compose up -d
docker compose ps
docker compose logs -f n8n
```

`up -d` starts all three containers in the background ("detached"). `ps` lists them; all should say `running` (Postgres also `healthy`). `logs -f` follows n8n's log; press Ctrl + C to stop watching (n8n keeps running). The first start takes a minute or two while Caddy fetches your certificate.

Open `https://n8n.example.com`. You should see the padlock and the owner setup screen. Create the owner account straight away, before anyone else finds the page, and turn on 2FA (Chapter 4).

5.10 Everyday commands

TO DO THIS	RUN THIS (IN `~/N8N-COMPOSE`)
See what's running	`docker compose ps`
Read n8n's recent log	`docker compose logs --tail=100 n8n`
Restart n8n	`docker compose restart n8n`
Stop everything	`docker compose down`
Start everything	`docker compose up -d`
Apply `.env` changes	`docker compose up -d`

5.11 Other ways to self-host

- **n8n's Caddy repository.** n8n's Hetzner and DigitalOcean guides clone `github.com/n8n-io/n8n-docker-caddy`, a ready-made Caddy setup using SQLite. It's a good alternative if you'd rather not write files by hand.
- **Traefik.** n8n's Docker Compose server guide uses Traefik, another reverse proxy that obtains certificates automatically, configured with labels in the Compose file.
- **The n8n-hosting repository** (`github.com/n8n-io/n8n-hosting`) has more Compose examples, including a hardened PostgreSQL setup with a separate task runner container.

- **npm.** `npm install n8n -g` then `n8n start` still works with Node.js 20.19–24, but npm installs are deprecated from n8n 3.0 and don't support n8n Assistant. Prefer Docker.
- **Testing webhooks locally.** n8n's docs describe a development tunnel using cloudflared. It's for local testing only, never production.

5.12 If it doesn't work

SYMPTOM	LIKELY CAUSE	FIX
Browser can't reach the site	DNS not updated yet, or firewall	Wait; check `ping`; check `sudo ufw status`
Certificate error	Port 80 blocked, or DNS points elsewhere	Open port 80; fix the A record; check `docker compose logs caddy`
n8n keeps restarting	Database not ready or wrong password	Check `docker compose logs n8n`; compare `.env` values
Webhook URLs show `localhost`	`N8N_WEBHOOK_URL` missing	Add it, then `docker compose up -d`
"Credentials could not be decrypted"	Encryption key changed	Restore the original `N8N_ENCRYPTION_KEY`

- [ ] My server has a domain name pointing at it
- [ ] The firewall allows only SSH, 80 and 443
- [ ] `.env` holds my secrets, and a copy is in my password manager
- [ ] n8n opens over HTTPS with a padlock
- [ ] I created the owner account and enabled 2FA immediately
- [ ] I've read Chapter 26 on backups and updates

CHAPTER 6

A Tour of the Editor

The n8n editor is where you'll spend most of your time. It looks busy at first, but there are only a few areas, and each has one job. This chapter walks you around them, shows the shortcuts that save the most time, and explains the three-panel window you'll use to set up every node.

IN THIS CHAPTER

- The Overview page, projects and folders
- The canvas, the nodes panel and the top bar
- The node details view: input, parameters, output
- Sticky notes, groups, and the command bar
- Keyboard shortcuts worth learning

6.1 The Overview page

When you sign in, you land on **Overview**: a list of everything you can access. Tabs along the top switch between **Workflows**, **Credentials**, **Executions** and **Data tables**.

- **Personal space** holds the workflows and credentials you created.
- **Projects** group workflows, credentials and data tables for a team. The number of shared projects depends on your plan.
- **Folders** let you organise workflows inside a project. On self-hosted Community, folders need the free registration.
- **Tags** are labels you add to workflows, like `marketing` or `client-acme`, then filter by.

6.2 The main areas

- **Left sidebar**: Overview (all workflows), Templates, Credentials, and Settings.
- **Canvas**: the big grid where you place and connect nodes. Scroll to zoom; drag the background to pan.
- **Top bar**: the workflow's name, the **Publish** button, and the **Editor / Executions** tabs.
- **Execute workflow** button (bottom of canvas): runs the whole workflow once, right now, for testing.

6.3 Adding nodes

Click the **+** on the canvas (or press `N`) to open the **nodes panel**. Search by app ("Gmail") or by action ("send email"). Nodes are grouped as:

- **Triggers**: how a workflow starts.
- **App nodes**: actions in other services (Google Sheets, Slack, Notion, Airtable...).
- **Core nodes**: logic and data: If, Switch, Edit Fields (Set), Merge, Loop Over Items, Wait, Code, HTTP Request.
- **AI nodes**: AI Agent, chat models, memory, tools, vector stores.

Many app nodes contain dozens of **operations**. The Google Sheets node, for example, can append rows, update rows, read rows, delete rows and more. Pick the node, then choose the operation.

6.4 Connecting nodes

Every node has small circles called **connectors**. Drag from the output connector on the right side of one node to the input connector on the left of another to join them. To insert a node between two others, hover over the line joining them and click the **+** that appears. To delete a connection, hover over it and click the bin icon.

6.5 The node details view

Double-click any node to open it. You'll see three panels:

PANEL	WHAT IT SHOWS
Input (left)	The data arriving from the previous node
Parameters (middle)	The node's settings: which account, which operation, which fields
Output (right)	What the node produced after you click **Execute step**

Switch the input and output views between **Table**, **JSON**, and **Schema**. Table is easiest to read; JSON shows the exact structure.

The middle panel has two tabs. **Parameters** holds what the node does. **Settings** holds how it behaves: retry on failure, what to do on error, whether to always output data, and notes to show on the canvas.

> **TIP**
> Test every node as you add it. Click **Execute step**, check the output looks right, then move on. Building in small tested steps is the secret to workflows that work first time.

6.6 Fixed values and expressions

Every parameter can be **Fixed** (a plain value you type) or an **Expression** (a formula that uses data from earlier nodes). Hover over a field to see the **Fixed / Expression** switch. A shortcut: type `=` in an empty field to switch it to expression mode. Chapter 9 covers expressions in depth.

6.7 Sticky notes and groups

Sticky notes are coloured notes you place on the canvas to explain what a section does. Press `Shift` + `S` to add one. They support Markdown, so you can add headings, lists and links. **Groups** box related nodes together: select nodes and press `Ctrl`/`Cmd` + `G`.

> **TIP**
> Future-you is a different person. A one-line sticky note (“Runs at 7am, emails Sam the weather”) saves twenty minutes of puzzling six months from now.

6.8 The command bar

Press `Ctrl`/`Cmd` + `K` to open the **command bar**. Type what you want: add a node, publish, open a workflow, open credentials, jump to settings, or retry an execution. It’s the fastest way around n8n once you know it exists.

6.9 Keyboard shortcuts worth learning

SHORTCUT	DOES
`N`	Open the nodes panel
`Ctrl`/`Cmd` + `Enter`	Execute the workflow
`Shift` + `P`	Publish

Ctrl / Cmd + Z	Undo (add Shift to redo)
Ctrl / Cmd + K	Command bar
1	Zoom to fit the whole workflow
0	Reset zoom
Space + drag	Move around the canvas
Shift + S	Add a sticky note
F2	Rename the selected node
D	Deactivate (skip) the selected node
P	Pin the selected node's data
Delete	Delete the selected node
Ctrl / Cmd + C / V	Copy and paste nodes

> **NOTE**
> Copied nodes are plain JSON text. You can paste them into a message to a colleague or a forum post, and they can paste them straight onto their canvas.

6.10 The Executions tab

Next to **Editor** at the top of each workflow is **Executions**: a list of every run of that workflow, with its status and timing. Click one to see exactly what each node received and produced. Chapter 18 shows how to use it for debugging.

6.11 Settings you'll visit

PLACE	WHAT'S THERE
Settings › Personal	Your name, password and 2FA
Settings › Usage and plan	Plan, licence, free registration
Settings › Community nodes	Install extra nodes (self-hosted)
Settings › Instance-level MCP	Let AI apps use your n8n (Chapter 22)
Workflow menu (…) › Settings	Error workflow, time zone, what executions to save

- [ ] I can open the nodes panel and add a node
- [ ] I can connect, insert and delete connections
- [ ] I know the three panels of the node details view

CHAPTER 7

Your First Workflow: A Morning Weather Email

Let's build a workflow that checks the weather every morning and emails you a summary. It uses a free weather service, Open-Meteo, that needs no account. Along the way you'll use a trigger, an HTTP request, an expression and an app node: the four skills behind almost every automation.

IN THIS CHAPTER

- Build a complete scheduled workflow, step by step
- Read data from a web service with HTTP Request
- Write your first expression
- Add a decision with the If node
- Publish it and check that it ran

BUILD THE WEATHER WORKFLOW

1. **Create a workflow.**
   On the Overview page click **Create Workflow**. Name it "Morning weather".

2. **Add a Schedule Trigger.**
   Click **Add first step**, search **Schedule**, and choose **Schedule Trigger**. Set **Trigger Interval** to **Days** and **Trigger at Hour** to **7am**. Close the node.

3. **Add an HTTP Request node.**
   Click the **+** after the trigger, search **HTTP Request**, and paste this into **URL** (it's New York; change the latitude and longitude to your city):

   ```
   TEXT

   https://api.open-meteo.com/v1/forecast?
   latitude=40.71&longitude=-74.01&daily=temperature_2m_max,temperature_2m_min,precipitatio
   n_probability_max&temperature_unit=fahrenheit&timezone=auto
   ```

   Click **Execute step**. The output shows today's forecast in a `daily` section.

4. **Add an Edit Fields (Set) node** to make a friendly message. Add a field named `message` and switch its value to **Expression** mode, then enter:

   ```
   TEXT

   Today: high {{ $json.daily.temperature_2m_max[0] }}°F, low {{
   $json.daily.temperature_2m_min[0] }}°F, {{ $json.daily.precipitation_probability_max[0]
   }}% chance of rain.
   ```

   Click **Execute step** and check the message reads correctly.

5. **Add a Gmail node** (or Outlook, or Send Email). Choose the **Send a message** operation. Under **Credential**, choose **Create new credential** and sign in with Google. Set **To** to your address, **Subject** to `Your morning weather`, and drag the `message` field from the input panel into **Message**.

6. **Test the whole thing.**
   Click **Execute workflow** and check your inbox.

7. **Publish.**
   Click **Publish** (top right). From tomorrow, it runs every morning at 7.

Congratulations: you've built a real automation. Notice the pattern you'll use forever: **trigger → get data → reshape data → send it somewhere.**

> **NOTE**
> The `[0]` in the expression means "the first entry in the list," which is today. `[1]` would be tomorrow.

7.1 Finding your latitude and longitude

Open Google Maps or OpenStreetMap, right-click your town, and the coordinates appear (two numbers such as `51.51, -0.13` for London). The first is latitude, the second longitude. Prefer Celsius? Remove `&temperature_unit=fahrenheit` from the URL and change `°F` to `°C` in your message.

7.2 What just happened, node by node

NODE	INPUT	OUTPUT
Schedule Trigger	Nothing: it starts the run	One item with the time it fired
HTTP Request	That item	One item: the forecast as JSON
Edit Fields	The forecast	One item with a `message` field
Gmail	The message	One item describing the sent email

Each node received exactly one item, so each ran once. Open each node after an execution to see these panels for yourself.

7.3 Add a decision

Make it smarter: only send an umbrella warning if rain is likely.

ADD AN IF NODE

1. **Insert an If node** between Edit Fields and Gmail. Hover over the connecting line and click **+**.
2. **Set the condition.**
   Value 1: drag in `precipitation_probability_max` (or use the expression `{{ $('HTTP Request').item.json.daily.precipitation_probability_max[0] }}`). Operation: **Number › is greater than**. Value 2: `50`.
3. **Connect the outputs.**
   Connect the **true** output to your Gmail node with the subject "☔ Take an umbrella!", and add a second Gmail node on **false** for normal days, or leave false empty to send nothing.

The expression in step 2 uses `$('HTTP Request')`, which means "the output of the node called HTTP Request." That's how you reach back past the previous node to any earlier one. If you rename that node, n8n updates expressions that refer to it.

7.4 Check that it really ran

The morning after you publish, open the workflow and click the **Executions** tab. You should see one run at 7:00 with a green **Succeeded** status. Click it to replay exactly what happened. If it's red, click the failed node to read the error, and see Chapter 18.

> **TIP**
>
> Don't want to wait until tomorrow? Temporarily set the schedule to **Minutes** with **Minutes Between Triggers** set to `2`, publish, watch two runs appear in Executions, then set it back to 7am and publish again.

7.5 Make it yours

Try one of these changes on your own. Each takes under five minutes:

- Send to Slack or Telegram instead of email
- Add tomorrow's forecast with `[1]`
- Run only on weekdays (Weeks interval)
- Add the city name to the subject
- Send a different message if it will be hot
- Log each forecast to Google Sheets

- [ ] My workflow ran successfully with **Execute workflow**
- [ ] I understand what `$json` and `[0]` mean
- [ ] I added an If node with true and false paths
- [ ] I published it and saw a production run in Executions

CHAPTER 8

How Data Flows: Items, JSON and Binary

Every confusing moment in n8n comes down to one question: *what data does this node actually have?* This chapter gives you a clear picture of how n8n packages data, how it moves between nodes, how files travel alongside it, and how to inspect it at every step.

IN THIS CHAPTER

- JSON in five minutes
- The shape of n8n data: a list of items, each with `json` and `binary`
- How nodes process many items
- Files (binary data) and the nodes that handle them
- Splitting, combining and linking items
- Pinning and mocking data while you build

8.1 JSON in five minutes

JSON (JavaScript Object Notation) is a way of writing structured data as text. It has just a few building blocks:

BUILDING BLOCK	LOOKS LIKE	HOLDS
String	`"Sam"`	Text, always in double quotes
Number	`42` or `3.5`	Numbers, no quotes
Boolean	`true` / `false`	Yes or no
Null	`null`	“Nothing here”
Object	`{ "name": "Sam" }`	Named fields (keys and values)
Array	`[ "a", "b", "c" ]`	An ordered list, counted from 0

Objects and arrays can sit inside each other as deeply as you like:

JSON

```
{
  "name": "Sam Lee",
  "email": "sam@example.com",
  "address": { "city": "Austin", "zip": "78701" },
  "orders": [ 120, 45, 300 ]
}
```

To reach a value you follow the path: `address.city` is `"Austin"`, and `orders[0]` is `120`, the first entry.

8.2 The shape of n8n data

All data passed between nodes is a **list of items**. Each item is an object with a `json` key holding the actual data, and optionally a `binary` key holding files:

JSON

```
[
  { "json": { "name": "Ana", "email": "ana@example.com" } },
  { "json": { "name": "Ben", "email": "ben@example.com" } }
]
```

That’s two items. In the editor’s Table view you’d see two rows. In JSON view n8n usually hides the `json` wrapper to keep things tidy, but it’s always there underneath.

> **TIP**
> In the Code node you can return plain objects, and n8n adds the `json` wrapper and the surrounding list for you. When something says “item,” think “one row” or “one record.”

8.3 How nodes process many items

Most nodes run their operation **once per item**. If a Google Sheets node outputs 50 rows and you connect a Slack node, Slack sends 50 messages. Each time, expressions like `{{ $json.name }}` resolve to the current item's value.

A few nodes are exceptions and work on the whole list at once, for example the Code node in **Run Once for All Items** mode, Aggregate, Summarize and Merge. Some database and API operations also run only once regardless of item count. The node's documentation says so when that's the case.

> **NOTE**
> Sub-nodes attached to AI nodes (chat models, memory, tools) behave differently: an expression in a sub-node always resolves to the **first** item.

8.4 Inspecting data

Every node's input and output panel has three views:

- **Table**: rows and columns, like a spreadsheet. Nested data shows in bold; click to expand.
- **JSON**: the exact structure. Best when you need precise paths.
- **Schema**: a list of field names and types, without values. Best for dragging fields into parameters.

Above the panel, n8n shows how many items there are, for example *3 items*. Check this number whenever results surprise you.

8.5 Files: binary data

Binary data is anything that isn't text: a PDF invoice, a photo, a spreadsheet file, an audio clip. n8n keeps files in the item's `binary` section, under a name such as `data` or `attachment_0`:

JSON

```
{
  "json": { "subject": "Your invoice" },
  "binary": {
    "data": {
      "fileName": "invoice-1042.pdf",
      "mimeType": "application/pdf",
      "fileExtension": "pdf"
    }
  }
}
```

The **MIME type** describes the kind of file (`image/png`, `application/pdf`). In the output panel, binary items show a **View** button so you can preview or download them.

NODE	WHAT IT DOES WITH FILES
Extract From File	Reads CSV, XLSX, PDF, HTML, JSON, ICS, RTF or text into JSON fields
Convert to File	Turns JSON into a CSV, XLSX, HTML, ICS, JSON or text file
Read/Write Files from Disk	Reads or saves files on a self-hosted server
Compression	Zips and unzips files
Edit Image	Resizes, crops and annotates images
HTTP Request	Downloads files when the response is a file

When a node like Gmail or Google Drive needs a file, it asks for the **binary property name** (often `data`). That's simply which file slot to use.

> **TIP**
> Email nodes can download attachments as binary data, usually in numbered slots such as `attachment_0`. Look at the node's output panel to see the exact names, then use that name wherever the next node asks for a binary property.

8.6 Splitting and combining items

Sometimes data arrives in the wrong shape. Two nodes fix most cases:

- **Split Out** turns one item containing a list into many items. One order with five products becomes five items, one per product.
- **Aggregate** does the reverse: many items become one item with a list, handy for building a single summary email.

Others you'll use: **Summarize** (totals and counts, like a pivot table), **Sort**, **Limit** (keep only the first *n*), and **Remove Duplicates**.

TURN ONE LIST INTO MANY ITEMS

1. **Start with one item** that has an `orders` array (pin some sample data as described below).
2. **Add Split Out.**
   Set **Fields To Split Out** to `orders`.
3. **Execute the step.**
   The output now shows one item per order, ready for per-order processing.

8.7 Item linking

n8n remembers which input item produced each output item. This **item linking** (also called *paired items*) is what lets `{{ $('Get Customer').item.json.name }}` find the right customer for the current order, even several nodes later. You rarely need to think about it. If you see an error about paired items, it usually means a Code node changed the number of items; Chapter 10 explains how to fix that.

8.8 Pinning and mocking data

Pinning freezes a node's output so later runs reuse it instead of calling the real service. Open a node after it has run and click the **pin** icon in the output panel, or select the node and press `P`.

You can also **mock** data: click the edit (pencil) icon on an output panel and type or paste your own JSON. This is perfect for building a workflow before the real data source exists.

> ☆ IMPORTANT
>
> Pinned data is for building only. Production executions ignore pins and always fetch fresh data, so a published workflow behaves correctly even if you forget to unpin.

- [ ] I can read a JSON object and follow a path like `address.city`
- [ ] I know n8n data is a list of items, each with `json` and maybe `binary`
- [ ] I check the item count when results surprise me
- [ ] I know Split Out and Aggregate change the number of items
- [ ] I can pin and mock data while building

CHAPTER 9

Expressions: Little Formulas, Big Power

An **expression** is a small formula inside a node's parameter that fills in a value when the workflow runs. "Send this email to *whoever filled in the form*." "Name the file *today's date*." If you've used formulas in a spreadsheet, you already understand the idea. This chapter teaches you to read and write them with confidence.

IN THIS CHAPTER

- How to switch a field to expression mode and map by dragging
- Reaching data from the current item and from earlier nodes
- Built-in variables: dates, workflow and execution details
- Handy functions for text, numbers and lists
- Working with dates using Luxon
- Getting help from AI and fixing common errors

9.1 Mapping by dragging

The easiest way to use data from an earlier node: open a node, find the field in the **Input** panel, and **drag it** into a parameter. n8n writes the expression for you.

9.2 Expression mode

Every parameter can be **Fixed** or **Expression**. Click **Expression** above a field (or type `=` in an empty one) and the field turns into an editor that shows a live **preview** of the result underneath. Anything inside `{{ }}` is calculated; anything outside is plain text:

```text
Hello {{ $json.firstName }}, your order #{{ $json.orderId }} has shipped!
```

> **TIP**
> Watch the preview as you type. If it shows the value you expect for the first item, you're nearly always right. If it says `undefined`, the path is wrong: check spelling and capitals.

9.3 Expressions cheat sheet

Expressions live inside double curly braces and can contain small bits of JavaScript.

EXPRESSION	GIVES YOU
`{{ $json.email }}`	The `email` field of the current item
`{{ $json.name.toUpperCase() }}`	The name in capital letters
`{{ $('Google Sheets').item.json.total }}`	A field from a specific earlier node, by name
`{{ $now.toFormat('yyyy-MM-dd') }}`	Today's date, like `2026-10-15`
`{{ $today.minus(7, 'days') }}`	The date one week ago
`{{ $json.price * 1.08 }}`	Arithmetic: price plus 8% tax
`{{ $json.status === 'paid' ? 'Thanks!' : 'Reminder' }}`	Choose text based on a condition

> **TIP**
> Not sure how to write an expression? Click **Ask AI** or the AI assistant in the expression editor and describe what you want: "the first name from the full_name field."

9.4 Reaching your data

WRITE THIS	TO GET
`$json.field`	A field of the current item
`$json.address.city`	A nested field
`$json.tags[0]`	The first entry of a list
`$json['First Name']`	A field whose name has spaces
`$('Node Name').item.json.field`	The matching item from an earlier node
`$('Node Name').first().json.field`	The first item from an earlier node
`$('Node Name').all()`	Every item from an earlier node, as a list

`$input.first().json.field`	The first item arriving at this node
`$input.all().length`	How many items arrived

`.item` is the clever one: it follows item linking (Chapter 8) to find the item that belongs to the one you're processing now. Use `.first()` when an earlier node produced a single item, such as a settings lookup.

9.5 Built-in variables

VARIABLE	GIVES YOU
`$now`	The current date and time
`$today`	Today at midnight
`$workflow.name` / `$workflow.id`	This workflow's name and ID
`$workflow.active`	Whether it's published
`$execution.id`	This run's ID, handy in error messages
`$execution.mode`	`test` (you clicked a button) or `production`
`$itemIndex`	The position of the current item, from 0
`$vars.NAME`	A custom variable (paid plans)

9.6 Helper functions

n8n adds its own conveniences on top of JavaScript:

FUNCTION	EXAMPLE	RESULT
`$if()`	`{{ $if($json.vip, 'Gold', 'Standard') }}`	Picks one of two values
`$ifEmpty()`	`{{ $ifEmpty($json.nickname, $json.name) }}`	A fallback when a value is empty
`$jmespath()`	`{{ $jmespath($json, 'orders[*].id') }}`	Queries JSON with JMESPath

9.7 Functions for text, numbers and lists

Add these after a value with a dot. Type a dot in the expression editor to see suggestions.

FUNCTION	ON	EXAMPLE	GIVES
`.toTitleCase()`	Text	`"ana lópez".toTitleCase()`	`Ana López`
`.extractEmail()`	Text	`"Mail: a@b.com".extractEmail()`	`a@b.com`
`.extractDomain()`	Text	`"a@shop.com".extractDomain()`	`shop.com`

`.removeTags()`	Text	`"<b>Hi</b>".removeTags()`	`Hi`
`.removeMarkdown()`	Text	`"**Hi**".removeMarkdown()`	`Hi`
`.isEmpty()`	Text, list	`"".isEmpty()`	`true`
`.toNumber()`	Text	`"42".toNumber()`	`42`
`.urlEncode()`	Text	`"a b".urlEncode()`	`a%20b`
`.toDateTime()`	Text	`"2026-10-16".toDateTime()`	A date you can format
`.sum()`	List	`[1,2,3].sum()`	`6`
`.average()`	List	`[2,4].average()`	`3`
`.unique()`	List	`[1,1,2].unique()`	`[1,2]`
`.pluck()`	List	`$json.people.pluck('email')`	Just the emails
`.first()` / `.last()`	List	`[5,6,7].last()`	`7`
`.join()`	List	`['a','b'].join(', ')`	`a, b`
`.chunk()`	List	`[1,2,3,4].chunk(2)`	`[[1,2],[3,4]]`

9.8 Dates and times with Luxon

n8n's `$now` and `$today` are **Luxon** date objects. Luxon is a popular JavaScript date library, and it makes date maths readable:

EXPRESSION	RESULT (EXAMPLE)
`{{ $now.toFormat('d LLLL yyyy') }}`	`16 October 2026`
`{{ $now.toFormat('cccc') }}`	`Friday`
`{{ $now.toFormat('HH:mm') }}`	`07:00`
`{{ $now.plus(1, 'day').toFormat('yyyy-MM-dd') }}`	Tomorrow's date
`{{ $today.startOf('month').toISO() }}`	The first of this month
`{{ $json.created.toDateTime().toFormat('d LLL') }}`	Format a date from your data

Common format codes: `yyyy` year, `MM` month number, `LLL` short month name, `LLLL` full month name, `dd` day, `cccc` weekday, `HH` hour (24h), `mm` minutes.

> **NOTE**
>
> Dates use the workflow's time zone if one is set in workflow settings, otherwise the instance time zone. If a time is an hour out, that's usually the reason.

9.9 Expressions or nodes?

You can do a lot inside expressions, but long formulas are hard to read. A good rule:

- **Short and specific** (format a name, pick a field): use an expression.
- **Reshaping many fields**: use **Edit Fields (Set)**.
- **Logic across many items** (totals, grouping, complex rules): use **Aggregate**, **Summarize**, or a **Code** node (Chapter 10).

9.10 Worked example: a friendly order summary

Given an item like `{ "firstName": "ana", "items": [ {"price": 12}, {"price": 30} ], "date": "2026-10-16" }`, this expression:

TEXT

```
Hi {{ $json.firstName.toTitleCase() }}, your {{ $json.items.length }} items
total ${{ $json.items.pluck('price').sum() }}, ordered on
{{ $json.date.toDateTime().toFormat('d LLLL') }}.
```

produces *Hi Ana, your 2 items total $42, ordered on 16 October.*

PROMPT

I'm writing an n8n expression. My item looks like this: [paste the JSON from the input panel]. I want [describe the result, e.g. "the customer's first name in capitals, or 'friend' if it's empty"]. Give me the expression inside {{ }} and explain each part.

9.11 Common expression errors

MESSAGE OR SYMPTOM	CAUSE	FIX
Preview shows `undefined`	Wrong path or capitalisation	Drag the field in instead of typing
Preview reports invalid syntax	A missing bracket or quote	Count your `(`, `)`, `'` and `}}`
"Can't get data for expression"	The referenced node hasn't run yet	Execute the earlier node first
Paired item error	Item links broken by a Code node	Use `.first()` or fix the Code node (Chapter 10)
Shows `[object Object]`	You inserted a whole object into text	Pick a field, or wrap it in `JSON.stringify()`

- [ ] I can switch a field to expression mode and read the preview
- [ ] I can reach a field from an earlier node with `$('Node Name')`
- [ ] I can format today's date with Luxon
- [ ] I know a few text and list functions
- [ ] I know when to use a node instead of a long expression

CHAPTER 10

The Code Node: JavaScript and Python

Most workflows never need code. But sometimes you need one small calculation that no node offers, or a tricky reshaping of data. The **Code** node lets you write a few lines of JavaScript or Python as a step. You don't need to be a programmer: with the examples here, and an AI assistant to help, a beginner can use it safely.

IN THIS CHAPTER

- When to reach for code, and when not to
- The two modes: once for all items, or once for each item
- What your code receives and must return
- Five copy-paste JavaScript recipes
- Python in n8n and how it differs
- Limits on Cloud and self-hosted, and getting AI help

10.1 When to use code

USE A CODE NODE WHEN...	USE OTHER NODES WHEN...
A calculation has many steps	You only need to rename or add fields (Edit Fields)
You need custom logic across all items	You need to branch (If, Switch)
No node does the transformation	A node or expression already does it
You're parsing an odd text format	You need HTTP calls or files (HTTP Request, Read/Write Files)

> **NOTE**
> The Code node can't make HTTP requests or read files on disk. Use the HTTP Request and Read/Write Files from Disk nodes for those, and pass their results into your code.

10.2 The two modes

MODE	RUNS	YOU USE
Run Once for All Items (default)	One time, with every item	`$input.all()`
Run Once for Each Item	Once per item	`$json` or `$input.item`

Choose **Each Item** for simple per-row changes. Choose **All Items** when you need to look across rows: totals, sorting, grouping, or comparing.

10.3 What goes in and what comes out

Your code receives items and must **return** items. Each returned item is an object with a `json` key (n8n will add it if you forget, and wrap a single object in a list for you). This minimal example, in **Run Once for All Items** mode, adds a field to every item:

ADD A FIELD TO EVERY ITEM

```
const items = $input.all();

for (const item of items) {
  item.json.fullName = `${item.json.first} ${item.json.last}`;
}

return items;
```

The same thing in **Run Once for Each Item** mode is even shorter:

PER-ITEM VERSION

```
const j = $input.item.json;
return { json: { ...j, fullName: `${j.first} ${j.last}` } };
```

The `...j` copies all existing fields, then `fullName` is added.

10.4 Five JavaScript recipes

1. Total an order list (All Items mode). Returns one item with the sum:

JAVASCRIPT

```javascript
const total = $input.all()
  .reduce((sum, item) => sum + Number(item.json.amount || 0), 0);

return [{ json: { total, count: $input.all().length } }];
```

2. Keep only recent items (All Items mode). Drops anything older than seven days:

JAVASCRIPT

```javascript
const cutoff = Date.now() - 7 * 24 * 60 * 60 * 1000;

return $input.all().filter(
  (item) => new Date(item.json.date).getTime() >= cutoff
);
```

3. Group by a field (All Items mode). Turns orders into one item per customer:

JAVASCRIPT

```javascript
const groups = {};
for (const { json } of $input.all()) {
  groups[json.customer] ??= { customer: json.customer, orders: 0 };
  groups[json.customer].orders += 1;
}
return Object.values(groups).map((g) => ({ json: g }));
```

4. Clean up messy text (Each Item mode):

JAVASCRIPT

```javascript
const raw = $input.item.json.phone || '';
const digits = raw.replace(/\D/g, '');
return { json: { ...$input.item.json, phone: digits } };
```

5. Build an HTML table for an email (All Items mode):

JAVASCRIPT

```
const rows = $input.all().map(({ json }) =>
  `<tr><td>${json.name}</td><td>${json.status}</td></tr>`
).join('');

return [{ json: { html: `<table>${rows}</table>` } }];
```

> **TIP**
>
> Use `console.log(something)` while building. The output appears in your **browser's developer console** (press F12 and choose **Console**), which is handy for peeking inside variables.

10.5 Built-ins you can use in code

Everything from Chapter 9 works in JavaScript code too: `$input`, `$json`, `$('Node Name').all()`, `$now`, `$today`, `$workflow`, `$execution` and `$vars`. Type `$` in the editor to see suggestions.

10.6 Python

Choose **Python** from the language option on the Code node. n8n runs real Python ("native Python") through its **task runners**, separate processes that execute code safely away from the main n8n program. It works a little differently from JavaScript:

- In **Run Once for All Items** mode, your items are in `_items`. In **Run Once for Each Item** mode, the current item is `_item`.
- Use square brackets to reach fields: `item["json"]["name"]`, not `item.json.name`.
- Other n8n built-ins (like `$('Node')`) aren't available in native Python.

PYTHON, RUN ONCE FOR ALL ITEMS

```
for item in _items:
    name = item["json"].get("name", "")
    item["json"]["name_upper"] = name.upper()

return _items
```

> **NOTE**
>
> On n8n Cloud, Python Code nodes can't import any libraries, even from Python's standard library. On self-hosted n8n you can allow specific standard-library and third-party modules through the task runner configuration. The older in-browser Python (Pyodide) was removed in n8n 2.0, so old Python examples using `item.json.field` need updating.

10.7 Libraries in JavaScript

On n8n Cloud, the JavaScript Code node can use two extra modules: Node.js's `crypto` (for hashing and random IDs) and the `moment` date library. Self-hosted users can allow other built-in and npm modules through environment settings (see Chapter 26).

10.8 Keeping items linked

When your code returns the **same number of items in the same order**, n8n keeps item linking intact, and later nodes can still use `$('Earlier Node').item`. When you return fewer or more items (filtering, grouping, splitting), the links may break. If a later node complains about paired items, either reference earlier data with `.first()` or `.all()`, or include the fields you need in your returned items so later nodes don't have to look back.

10.9 Let AI write it

The Code node has AI help built in on n8n Cloud: describe what you want, and it drafts the code. Any chat assistant works too:

PROMPT

Write an n8n Code node in JavaScript, mode "Run Once for All Items".
Input items look like this: [paste one item's JSON]. I want the output to be [describe it]. Return an array of objects with a "json" key. Keep it short and add a comment on each line explaining what it does.

> ⚠ **WARNING**
>
> Read AI-written code before you run it on real data. Test with pinned sample data first, and check the output panel item by item.

10.10 Common Code node errors

ERROR	CAUSE	FIX
"Code doesn't return items properly"	Returned something that isn't a list of objects	Return `[{ json: {...} }]` or the items list
"Cannot read properties of undefined"	A field you used doesn't exist on some item	Use `item.json.x ?? ''` for a default
`$input.item` **is undefined**	Wrong mode	Use `$input.all()` in All Items mode
`Cannot find module`	Library not allowed on this instance	Use built-ins, or allow the module (self-hosted)
Python `KeyError`	A field is missing	Use `item["json"].get("field")`

- [ ] I know the difference between the two modes
- [ ] I can return items in the right shape
- [ ] I've tried at least one recipe with pinned data
- [ ] I know Python uses `_items` and square brackets
- [ ] I check AI-written code before trusting it

CHAPTER 11

Core Nodes: The Toolkit You'll Use Every Day

App nodes talk to other services; **core nodes** are the logic and plumbing that join everything together. Learn these ten or so and you can build almost any workflow. This chapter explains each one with a small example.

IN THIS CHAPTER

- Edit Fields (Set), Filter, If and Switch
- Merge and Loop Over Items
- HTTP Request, including pagination and cURL import
- Wait, and the smaller helpers
- A reference table of every everyday core node

11.1 Edit Fields (Set)

Edit Fields (often still called the Set node) adds, changes or removes fields. It has two modes: **Manual Mapping**, where you add fields one by one in the interface, and **JSON Output**, where you write the JSON yourself.

- Turn on **Keep Only Set Fields** (or set **Include in Output** accordingly) to drop everything you didn't list, which keeps later nodes tidy.
- Each field has a type (String, Number, Boolean, Array, Object). Choose the right one so later comparisons work.

> TIP
>
> Put an Edit Fields node right after a trigger to rename messy field names (`Your E-mail Address`) into clean ones (`email`). Every later node becomes easier to read.

11.2 Filter

Filter keeps only items that match your conditions and silently drops the rest. Use it to skip unneeded work early, which also saves executions downstream. Example: keep orders where `{{ $json.total }}` **is greater than** `100`.

11.3 If

If sends each item down a **true** or **false** output. Conditions compare values by type: String (contains, starts with, is empty…), Number (greater than…), Date & Time (is after…), Boolean, Array (contains, length…) and Object. Combine several conditions with **AND** (all must match) or **OR** (any may match).

11.4 Switch

Switch is an If with many outputs, like a railway junction. In **Rules** mode, you add one rule per output, for example "category is *billing*," "category is *technical*." Useful options:

- **Fallback Output**: where items go if no rule matches. Add an extra output so nothing disappears silently.
- **Send data to all matching outputs**: otherwise an item only goes to the first matching rule.
- **Ignore Case**: treat `Billing` and `billing` the same.

Expression mode lets an expression return the output number directly.

11.5 Merge

Merge waits for two (or more) inputs and combines them. Choose a mode:

MODE	DOES	LIKE
Append	Lists all items from input 1, then input 2	Stacking two piles
Combine › Matching Fields	Joins items where a field matches (e.g. `email`)	A spreadsheet lookup
Combine › Position	Pairs item 1 with item 1, item 2 with item 2	Zipping two lists
Combine › All Possible Combinations	Every item with every other item	A grid
SQL Query	Write SQL over the inputs	A database query
Choose Branch	Waits for both, outputs one	A gate

When matching fields, choose what to keep: **Keep Matches** (inner join), **Keep Everything** (outer join), **Enrich Input 1** (keep all of input 1, add matches from input 2) and so on.

> **NOTE**
> A common pattern is If → two branches → Merge. Merge waits for both branches, so the workflow continues once, with all the data, after the paths split.

11.6 Loop Over Items (Split in Batches)

Usually you **don't need a loop**: nodes already process every item. Use **Loop Over Items** when you must handle items in smaller groups, typically to respect an API's **rate limit** (how many requests per minute it accepts).

PROCESS 500 CONTACTS, 10 AT A TIME

1. **Add Loop Over Items** after the node producing the contacts. Set **Batch Size** to `10`.
2. **Connect the loop output** to the work you want to do (for example an HTTP Request), then a **Wait** node of 2 seconds.
3. **Connect the Wait node back** to the Loop Over Items input. That closes the loop.
4. **Connect the done output** to whatever should happen at the end. It receives all processed items once every batch has finished.

You can also build a "loop until" by connecting a later node back to an earlier one with an If node deciding when to stop.

11.7 HTTP Request

HTTP Request calls any web API, so it's your escape hatch when n8n has no dedicated node. The key settings:

SETTING	WHAT TO PUT
Method	`GET` to read, `POST` to create, `PUT` / `PATCH` to update, `DELETE` to remove
URL	The API endpoint from the service's docs
Authentication	A **predefined credential** for a known app, or a **generic** one (header, bearer, basic, OAuth2)
Send Query Parameters	Values after `?` in the URL
Send Headers	Extra headers the API asks for
Send Body	JSON, form data, or a binary file for POST/PUT

Useful options: **Batching** (items per batch plus a pause, for rate limits), **Pagination** (fetch all pages automatically, either by updating a parameter each request or by following a "next URL" in the response), **Response › Never Error** (don't fail on 4xx/5xx codes), and **Timeout**.

> **TIP**
>
> API docs often show a `curl` command. Click **Import cURL** in the HTTP Request node, paste it, and n8n fills in method, URL, headers and body for you. Move any secret key into a credential afterwards.

> **TIP**
>
> Choosing **Predefined Credential Type** lets you reuse, say, your Notion credential to call a Notion API endpoint the Notion node doesn't cover.

11.8 Wait

Wait pauses a workflow. Its modes are **After Time Interval** (wait 10 minutes), **At Specified Time** (wait until Monday 9am), **On Webhook Call** (wait until something calls a unique "resume" URL, available as `$execution.resumeUrl`) and **On Form Submitted** (wait until someone completes a form). Long waits don't hold up your server: n8n saves the execution and resumes it later.

11.9 Everyday helpers

NODE	USE IT TO
Aggregate	Combine many items into one item with lists
Split Out	Turn a list inside one item into many items
Summarize	Count, sum or average, grouped by a field
Sort / Limit	Order items; keep only the first *n*
Remove Duplicates	Drop repeats in this run, or items seen in previous runs
Compare Datasets	Find what's new, changed or missing between two lists
Date & Time	Format, add to, or compare dates
Crypto	Hash values, generate random strings
HTML	Extract content from web pages with CSS selectors
Respond to Webhook	Send a custom reply to a Webhook caller
Execute Sub-workflow	Call another workflow (Chapter 14)
Stop and Error	Fail on purpose with a clear message (Chapter 17)
No Operation, do nothing	A placeholder that keeps the canvas tidy
Data Table	Read and write n8n's built-in tables (Chapter 15)

> **TIP**
>
> **Remove Duplicates** has a mode called **Remove Items Processed in Previous Executions**. It remembers what it has seen across runs, which is perfect for "only alert me about new RSS posts" without a database.

- [ ] I can reshape data with Edit Fields
- [ ] I can branch with If and Switch, and rejoin with Merge
- [ ] I know when I need Loop Over Items (and when I don't)
- [ ] I can call an API with HTTP Request and import a cURL command
- [ ] I know Remove Duplicates can remember previous runs

CHAPTER 12

Triggers: How Workflows Start

Every workflow begins with a trigger. Picking the right one is the first design decision you make, and it affects how quickly your workflow reacts and how many executions it uses. This chapter covers each kind, with set-up steps for the most important.

IN THIS CHAPTER

- The trigger types at a glance
- Schedules, including Cron expressions
- Webhooks: URLs, security and responses
- Forms, multi-page forms and chat
- App triggers: instant versus polling
- Test URLs versus production URLs

TRIGGER	STARTS WHEN…	EXAMPLE
Schedule Trigger	A time arrives: every hour, every Monday at 9…	Weekly report
Webhook	Another app sends data to n8n's unique URL	Payment received in Stripe
n8n Form Trigger	Someone submits a form n8n hosts for you	Event sign-up
App triggers	Something happens in an app (new email, new row, new file)	New Gmail message with a label
Chat Trigger	Someone sends a message in a chat window	A chatbot for your website
Execute Workflow Trigger	Another workflow calls this one	Reusable "send notification" sub-workflow
Error Trigger	Another workflow fails	Alert yourself on failure
Manual	You click **Execute workflow**	Testing
MCP Server Trigger	An AI app calls one of its tools	Chapter 22

12.1 Schedule Trigger

Choose a **Trigger Interval**: Seconds, Minutes, Hours, Days, Weeks, Months, or **Custom (Cron)**. Each interval asks for the details you'd expect, such as **Trigger at Hour** and **Trigger at Minute**, or the weekdays for a weekly schedule. You can add several rules to one trigger.

Cron is a compact way to write schedules. n8n's Cron field accepts an optional seconds column at the front, which is why you may see six positions:

CRON EXPRESSION	MEANS
`0 9 * * 1-5`	9:00 every weekday
`*/15 * * * *`	Every 15 minutes
`0 8 1 * *`	8:00 on the 1st of each month
`0 0 18 * * 5`	18:00 every Friday (with seconds first)

The five standard positions are minute, hour, day of month, month and day of week. Sites like crontab.guru explain any expression in plain English.

> ☆ IMPORTANT
>
> Schedules only run when the workflow is **published**, and they use the workflow's time zone (set in workflow settings) or, failing that, the instance time zone. Self-hosted instances default to New York time.

> **TIP**
> Every schedule run is a production execution. A trigger every minute is about 43,000 runs a month, far beyond a Starter plan. Every 15 minutes is about 2,900. Pick the slowest schedule that does the job.

12.2 Webhook

A **webhook** is a web address that another service calls when something happens. The Webhook node gives you that address.

SET UP A WEBHOOK

1. **Add a Webhook node.**
   Choose the **HTTP Method** the other service will use, usually `POST`.
2. **Optionally set a Path**, such as `new-order`, instead of the random one. Paths can include parameters like `/order/:id`.
3. **Choose Authentication.**
   Basic auth, header auth or JWT auth stop strangers from triggering your workflow. Use one for anything public.
4. **Choose Respond.**
   Immediately replies "Workflow got started"; **When Last Node Finishes** replies with the final data; **Using 'Respond to Webhook' Node** lets you build a custom reply.
5. **Copy the Test URL**, click **Listen for test event**, and send a test from the other service. The data appears in the output.
6. **Publish**, then give the other service the **Production URL**.

Useful options include **IP(s) Allowlist** (only accept calls from listed addresses), **Ignore Bots**, **Allowed Origins (CORS)** for calls from web pages, and **Binary Property** to receive files. The maximum payload is 16 MB by default.

12.3 Test URLs versus production URLs

Webhook and Form triggers each have **two** addresses:

- **Test URL**: works while you click **Listen for test event** in the editor, so you can see data arrive.
- **Production URL**: works once the workflow is **published**.

On self-hosted n8n they differ only in the path: test URLs contain `/webhook-test/` and production URLs contain `/webhook/`.

The most common beginner puzzle, “my webhook works in testing but not for real,” is almost always because the workflow isn’t published, or the other app is still using the test URL.

12.4 n8n Form Trigger

n8n can host a simple web form for you, with no website needed.

BUILD A SIGN-UP FORM

1. **Add an n8n Form Trigger.**
   Give it a **Form Title** and **Form Description**.
2. **Add form elements.**
   Types include Text, Email, Number, Date, Dropdown, Checkboxes, Radio Buttons, Textarea, File, Password, Hidden Field and Custom HTML. Tick **Required Field** where needed.
3. **Choose Respond When.**
   Form Is Submitted confirms straight away; **Workflow Finishes** waits until the workflow has run.
4. **Test it.**
   Click **Execute step**; n8n opens the form. Submit it and the answers appear as fields named after each element.
5. **Publish and share the production URL.**

For **multi-page forms**, add **n8n Form** nodes after the trigger. Each is another page, and the last can show a completion screen, redirect to a web page, show your own text, or return a file. You can put If nodes between pages to show different questions depending on earlier answers.

> TIP
> Options let you hide the “Form automated with n8n” line, rename the submit button, set a custom path, and restyle the form with CSS.

12.5 Chat Trigger

The **Chat Trigger** starts a workflow from a chat message, usually feeding an AI Agent (Chapter 20). Turn on **Make Chat Publicly Available** to get a shareable chat page. **Mode** is either **Hosted Chat** (n8n’s ready-made chat page, set up with options for title, placeholder and welcome message) or **Embedded Chat** (your own website calls the workflow, for example with n8n’s chat widget). Authentication can be none, basic auth, or **n8n User Auth** (only people logged in to your n8n).

12.6 App triggers: instant or polling?

App triggers come in two flavours:

- **Instant (webhook-based)**: the app tells n8n immediately. Examples: Telegram, Stripe, GitHub. Needs a public HTTPS address, so it won't work on a laptop without a tunnel.
- **Polling**: n8n checks the app on a timer, set by **Poll Times**. Examples: Gmail Trigger, Google Sheets Trigger. Each check that finds something new is an execution.

> ⚠ **WARNING**
>
> Telegram allows only one webhook per bot. If you test a Telegram Trigger while the same workflow is published, the test URL replaces the production one, and the published workflow stops receiving messages until you publish again.

12.7 Other triggers

- **Execute Sub-workflow Trigger** (shown as "When Executed by Another Workflow"): Chapter 14.
- **Error Trigger**: Chapter 17.
- **RSS Feed Trigger**: new items in a feed.
- **Local File Trigger** (self-hosted): a file changes in a folder.

- [ ] I can choose between schedule, webhook, form, chat and app triggers
- [ ] I know schedules use the workflow or instance time zone
- [ ] I've secured any public webhook with authentication
- [ ] I give other services the production URL, not the test URL
- [ ] I know polling triggers use executions every time they find something

CHAPTER 13

Connecting Your Apps: Credentials and OAuth

To read your email or post to your Slack, n8n needs permission. That permission is stored as a **credential**. This chapter explains the main kinds, walks through the trickiest one (Google on a self-hosted server), and shows how to call any API.

IN THIS CHAPTER

- What credentials are and where they live
- OAuth versus API keys
- Setting up Google OAuth on self-hosted n8n
- Restricting which domains a credential may be used with
- Calling any API with HTTP Request
- Good security habits

13.1 Credentials

A **credential** stores the sign-in for an app so nodes can use it. Create them from a node (**Credential › Create new credential**) or from **Credentials** in the sidebar. There are two main kinds:

- **OAuth** ("Sign in with Google," "Connect to Slack"): click a button and approve in a pop-up. On n8n Cloud, most of these work instantly.
- **API key or token**: copy a secret key from the app's developer settings and paste it in.

When you save a credential, n8n tests it to confirm it works. Credentials are encrypted in n8n's database with your instance's encryption key, and their secret values are never shown again after saving.

13.2 How OAuth works, in plain English

OAuth is the "valet key" of the internet. Instead of giving n8n your Google password, you sign in to Google, and Google hands n8n a limited key that can do only what you approved (for example "read and send Gmail"). You can revoke that key at any time from your Google account without changing your password.

The flow has one fiddly part: the app must know where to send you back after you approve. That address is the **OAuth Redirect URL**, shown in every n8n OAuth credential. It must match exactly what's registered with the app.

13.3 Managed versus custom OAuth

On **n8n Cloud**, many popular apps offer **Managed OAuth2**: just click **Sign in with Google** and you're done. For Google, that covers Gmail, Sheets, Drive, Calendar, Docs, Slides, Contacts, Tasks, Chat and YouTube, plus their triggers.

On **self-hosted** n8n (and for apps without managed OAuth), you create your own **OAuth app** in the provider's developer console. Each credential page in n8n links to step-by-step instructions.

13.4 Walkthrough: Google OAuth on self-hosted n8n

CREATE A GOOGLE OAUTH CREDENTIAL

1. **Create a Google Cloud project.**
   Go to console.cloud.google.com, open the project dropdown and choose **New project**. Name it "n8n" and select **Create**.

2. **Enable the APIs you need.**
   Go to **APIs & Services › Library**, search for, say, **Gmail API**, and select **Enable**. Repeat for Sheets, Drive and so on.

3. **Configure the consent screen.**
   Go to **APIs & Services › OAuth consent screen** and select **Get started**. Enter an app name and support email. For **Audience**, choose **Internal** (Workspace users only) or **External** (any Google account). Accept the policy and create it.

4. **Add your domain.**
   Under **Branding › Authorized domains**, add your n8n domain (for example `example.com`).

5. **Create the client.**
   Go to **APIs & Services › Credentials**, choose **+ Create credentials › OAuth client ID**, and set **Application type** to **Web application**.

6. **Paste the redirect URL.**
   In n8n, create a Google credential (for example “Gmail OAuth2 API”) and copy its **OAuth Redirect URL**. Paste it under **Authorized redirect URIs** in Google, then select **Create**.

7. **Copy the ID and secret back.**
   Paste Google’s **Client ID** and **Client Secret** into the n8n credential, select **Sign in with Google**, approve, and **Save**.

> ⚠ **WARNING**
> With an **External** audience, your Google app starts in **Testing** mode: only Google accounts you add as **test users** can sign in. Everyone else sees “access denied.” Add yourself as a test user on the consent screen settings.

13.5 Restricting where a credential can be used

Many credentials have an **Allowed HTTP Request Domains** field. Set it to **Specific Domains** (for example `api.github.com`) so the credential can’t be sent to any other site from an HTTP Request node, or **None** to block HTTP Request use entirely. This protects you if someone edits a workflow carelessly.

13.6 Fixed and end-user credentials

For OAuth credentials, some plans let you choose a **credential type**: a **Fixed credential** (the same account whatever happens, the default) or an **End-user credential** (each person who triggers the workflow uses their own account, a preview feature on higher plans).

13.7 When there’s no node: HTTP Request

Almost every online service has an **API**. If n8n has no dedicated node for it, use the **HTTP Request** node with the service’s documentation. Many API docs show a `curl` command; n8n’s HTTP Request node has **Import cURL**, which fills in everything for you.

CALL AN API WITH A KEY

1. **Find the API docs.**
   Note the endpoint URL and how it wants the key, usually a header such as `Authorization: Bearer YOUR_KEY` or `X-API-Key: YOUR_KEY`.

2. **Create a generic credential.**
   In the HTTP Request node choose **Authentication › Generic Credential Type**, then **Header Auth** (or **Bearer Auth**). Enter the header name and value.

3. **Set method and URL**, add any query parameters, and click **Execute step**.
4. **Inspect the response.**
   If it fails, read the status code: 401 means the key is wrong, 403 not allowed, 404 wrong URL, 429 too many requests.

13.8 Sharing credentials

On plans with projects, a credential belongs to a project or to your personal space. Everyone in a project can **use** its credentials in workflows without seeing the secret values. Credential sharing with specific people requires a paid plan.

13.9 Security habits

- Name credentials clearly, for example “Gmail: personal” and “Gmail: business.”
- Give each credential the minimum access it needs.
- Never paste keys into Code nodes or text fields; always use credentials.
- Remove credentials you no longer use.
- Revoke an app’s access at the provider (Google, Slack…) if a credential might have leaked, then create a fresh one.

- [ ] I’ve created at least one OAuth and one API-key credential
- [ ] I know where to find the OAuth Redirect URL
- [ ] On self-hosted, I’ve added myself as a Google test user
- [ ] I’ve restricted credential domains where it makes sense
- [ ] No secrets live in my Code nodes or text fields

CHAPTER 14

Sub-workflows: Build Once, Reuse Everywhere

As you build more workflows, you'll notice the same steps appearing again and again: "send a Slack alert," "look up a customer," "log this to a sheet." A **sub-workflow** is a workflow that other workflows call, like a function in programming. Build it once, call it from anywhere, fix it in one place.

IN THIS CHAPTER

- Why sub-workflows make life easier
- Creating a sub-workflow with defined inputs
- Calling it with Execute Sub-workflow
- How data passes in and out
- Turning part of an existing workflow into a sub-workflow

14.1 Why bother?

- **Less repetition.** Change your alert format once, and every workflow that uses it updates.
- **Tidier canvases.** A 60-node workflow becomes a readable 10-node workflow calling helpers.
- **Lower memory use.** Big jobs split into smaller runs.
- **Cheaper.** Sub-workflow executions don't count towards your plan's monthly execution limit.
- **AI tools.** An AI agent can call a sub-workflow as a tool (Chapter 20).

14.2 Create the sub-workflow

BUILD A REUSABLE "NOTIFY ME" WORKFLOW

1. **Create a new workflow** called "Notify me".

2. **Add the Execute Sub-workflow Trigger.**
   In the trigger list it's titled **When Executed by Another Workflow**.

3. **Define the inputs.**
   Set **Input data mode** to **Define using fields below**, and add `title` (String) and `message` (String). Other modes are **Define using JSON example** and **Accept all data**.

4. **Add the work.**
   For example a Slack node posting `*{{ $json.title }}*` and `{{ $json.message }}` to your alerts channel.

5. **Make sure it has no errors.**
   A parent workflow can't call a sub-workflow that contains errors.

> **TIP**
> In the sub-workflow's settings, **This workflow can be called by** controls who may use it: for example only workflows in the same project, or specific workflows.

14.3 Call it from another workflow

CALL "NOTIFY ME"

1. **Add an Execute Sub-workflow node** where you want the notification.

2. **Choose the workflow** from the list (source **Database**). Its inputs, `title` and `message`, appear automatically.

3. **Fill in the inputs** with expressions, such as `New order` and `{{ $json.customer }} spent {{ $json.total }}`.

4. **Choose the mode.**
   Run once with all items sends all items in one call; **Run once for each item** calls it for every item.

5. **Run it.**
   Open the node afterwards and click **View sub-execution** to jump into the sub-workflow's run.

14.4 How data passes between workflows

1. The Execute Sub-workflow node sends its data to the sub-workflow's trigger.
2. The sub-workflow runs.
3. Whatever its **last node** outputs comes back to the Execute Sub-workflow node in the parent, and the parent carries on.

If you don't need the result, turn off the option **Wait for Sub-Workflow Completion** so the parent continues immediately, like posting a letter rather than waiting for a reply.

14.5 Convert part of a workflow

Already have a long workflow? n8n can convert a selection of connected nodes into a sub-workflow: it moves them into a new workflow and puts an Execute Sub-workflow node in their place. Search the docs for "Convert to sub-workflows" for the current steps. Check the new sub-workflow's inputs afterwards.

14.6 Ideas for your library

- Notify me (Slack, email or Telegram)
- Look up or create a CRM contact
- Log an event to a sheet or data table
- Summarise text with AI
- Format a date or currency your way
- Get an access token for an awkward API

- [ ] I've built a sub-workflow with defined inputs
- [ ] I've called it with Execute Sub-workflow and viewed the sub-execution
- [ ] I know the last node's output is returned to the parent
- [ ] I know sub-workflow runs don't count against my execution quota

CHAPTER 15

Data Tables: A Spreadsheet Inside n8n

Workflows often need to remember things between runs: which emails were already handled, a list of VIP customers, the prompt text your AI uses. You could use Google Sheets or a database, but n8n has **data tables** built in: simple tables that live inside your n8n project.

IN THIS CHAPTER

- What data tables are good (and not good) for
- Creating a table by hand or from a CSV
- Reading and writing rows with the Data Table node
- A worked example: never process the same item twice
- Limits and alternatives

15.1 Good uses

- Remembering what you've already processed, to avoid duplicates
- Lookup tables, such as product codes to names
- Reusable prompts and message templates
- A simple log of what your workflows did
- Test cases for checking AI workflows

15.2 Create a table

MAKE A "PROCESSED" TABLE

1. **Open your project** (or Overview) and select the **Data tables** tab.

2. **Select Create Data table** from the button in the top right.
3. **Name it** “processed_items”.
4. **Choose From scratch**, then add columns: `item_id` (string) and `processed_at` (date).

You can also choose **Import CSV** to build a table from a spreadsheet file, and later **Download CSV** from the table’s menu. In the table view you can add, edit, reorder and delete rows and columns by hand.

15.3 The Data Table node

In a workflow, the **Data Table** node works with two resources:

RESOURCE	OPERATIONS
Row	Insert, Get, Update, Upsert, Delete, If Row Exists, If Row Does Not Exist
Data table	Create, List, Update, Delete

Upsert means “update the row if it exists, otherwise insert it.” **If Row Does Not Exist** passes through only the items that aren’t in the table yet, which makes it a one-node duplicate filter.

15.4 Worked example: skip what you’ve seen

PROCESS EACH ITEM ONLY ONCE

1. **Start with your trigger and data**, for example an RSS Read node returning posts with a `link` field.
2. **Add a Data Table node**, resource **Row**, operation **If Row Does Not Exist**. Choose `processed_items` and match `item_id` against `{{ $json.link }}`.
3. **Do the work** on the items that come out: send them to Slack, summarise them, and so on.
4. **Record them.**
   Add another Data Table node, operation **Insert**, setting `item_id` to `{{ $json.link }}` and `processed_at` to `{{ $now.toISO() }}`.

Run it twice: the second run finds nothing new and stops early.

15.5 Limits

- By default, all data tables in an instance share **200 MiB** of storage. n8n warns at 80%. Past the limit, inserts and updates fail. Self-hosted owners can raise it with the `N8N_DATA_TABLES_MAX_SIZE_BYTES` environment variable.
- Tables belong to a project, and everyone in the project can use them.
- The Code node can't read data tables directly; use the Data Table node and pass the results in.

15.6 Data tables, variables or a real database?

NEED	USE
Rows and columns, moderate size	Data tables
A few constant values used everywhere (paid plans)	Custom variables (`$vars`)
Large data, many users, complex queries	PostgreSQL, Airtable, Google Sheets or similar

- [ ] I've created a data table by hand
- [ ] I've used Insert and If Row Does Not Exist in a workflow
- [ ] I know the 200 MiB default limit
- [ ] I know when to move to a real database

CHAPTER 16

Saving, Publishing and Versions

n8n separates *working on* a workflow from *running it for real*. Understanding this saves you from the two classic mistakes: “I changed it but nothing happened” and “I broke the live version while experimenting.”

IN THIS CHAPTER

- Autosave and drafts
- Publishing, unpublishing and the publish button’s states
- Workflow history: viewing and restoring old versions
- Naming versions
- Exporting, importing and sharing workflows as files
- Tags, folders and settings that help you stay organised

16.1 Autosave and drafts

Changes save automatically as you edit, usually within a few seconds; there’s no save button to remember. Every edit stays in **draft** until you publish. Only one person can edit a workflow at a time: others see it read-only until the editor becomes inactive.

16.2 Publishing

Publishing makes the current version live. Production executions then always use that published version, not your latest edits. When you publish:

- Webhook and Form triggers start answering on their production URLs.
- Schedules start running.
- App events start triggering the workflow.

PUBLISH A WORKFLOW

1. **Click Publish** in the top bar, or press Shift + P.
2. **Optionally name the version** and add a description of what changed.
3. **Click Publish** in the dialog. The button shows **Publishing** while n8n registers your triggers in the background.

The **Publish** button tells you the state at a glance:

BUTTON STATE	MEANING
Disabled	Nothing to publish
Active	Unpublished changes are waiting
Published	Live and up to date
Published, has changes	Live, but your newer edits aren't live yet
Published, partial	Some triggers failed to activate; hover to see which, then publish again
Failed to publish	No trigger activated; fix the problem and retry

To **unpublish** (stop a workflow from running), use the arrow next to **Publish**, the workflow list, or version history. Shortcut: Ctrl / Cmd + U.

> TIP
> Because edits stay in draft, you can safely improve a live workflow during the day and publish when you're happy. Test with **Execute workflow** first.

16.3 Workflow history

Each autosave creates a new **version**. Open history with the clock icon in the top bar. For each version you can:

- **Restore version**: make it your current draft (it doesn't change production until you publish);
- **Clone to new workflow**: start a copy from it;
- **Open version in new tab**: compare side by side;
- **Download**: save it as JSON;
- **Name version**: protect it from automatic cleanup.

How far back you can go depends on your plan: the last 24 hours for everyone, five days on Cloud Pro, and full history on Enterprise. Named versions are available on Cloud Pro and Enterprise and self-hosted Business and Enterprise, and are never pruned.

16.4 Exporting and importing

Workflows are JSON files, which makes them easy to back up and share.

- **Download:** workflow menu (…) › **Download**.
- **Import:** in a new workflow, use the workflow menu › **Import from File** or **Import from URL**, or simply **paste** the JSON onto the canvas.
- **Copy nodes:** select nodes, `Ctrl`/`Cmd` + `C`, and paste into another workflow or a message.

> ⚠ **WARNING**
> Exported workflows don't contain credential secrets, but they do contain node settings, which can include things like email addresses, sheet IDs or URLs. Check before posting a workflow publicly, and remove anything personal.

16.5 Staying organised

- **Names:** "Weekly sales report (Mon 8am)" beats "My workflow 7."
- **Tags:** add from the top bar, then filter the list by tag.
- **Folders and projects:** group related workflows.
- **Descriptions and sticky notes:** explain the why, not just the what.
- **Archive** workflows you no longer use rather than deleting them straight away.

16.6 Workflow settings worth knowing

Open the workflow menu (…) › **Settings**:

SETTING	USE IT TO
Error Workflow	Run an alert workflow when this one fails (Chapter 17)
This workflow can be called by	Control which workflows may use it as a sub-workflow
Timezone	Override the instance time zone for schedules and dates
Save failed / successful production executions	Choose what to keep in the Executions list
Save manual executions	Keep or discard your test runs
Timeout Workflow	Stop runs that take too long

16.7 Git and environments (advanced)

On self-hosted Business and Enterprise plans, **source control** connects n8n to a Git repository so you can keep separate development and production instances and move changes between them in a controlled way. Most individuals don't need this: version history and exported JSON files are plenty.

- [ ] I know edits stay in draft until I publish
- [ ] I can read the Publish button's state
- [ ] I've restored or cloned an older version
- [ ] I've exported a workflow as JSON and imported it again
- [ ] I've set a time zone and error workflow in workflow settings

CHAPTER 17

Handling Errors Gracefully

Things go wrong: an API is down for a minute, a customer leaves a field blank, a password expires. A reliable workflow expects this. This chapter shows the tools n8n gives you to retry, route around, and get told about problems before anyone else notices.

IN THIS CHAPTER

- Retry On Fail for flaky services
- On Error: stop, continue, or take an error branch
- Error workflows and the Error Trigger
- Failing on purpose with Stop and Error
- Patterns for missing data and rate limits

TOOL	WHAT IT DOES
Retry On Fail (node settings)	Automatically retry a flaky step, e.g. 3 times, waiting 5 seconds
On Error › Continue (node settings)	Keep the workflow going and send the error down a separate output
Error workflow	In workflow **Settings**, choose a workflow that starts with an **Error Trigger**; it runs whenever this one fails, so you can email or Slack yourself the details
Stop and Error node	Deliberately fail with a clear message when something's wrong

17.1 Retry On Fail

Open a node, switch to its **Settings** tab, and turn on **Retry On Fail**. Set **Max Tries** (for example 3) and **Wait Between Tries** (for example 5,000 ms). Use it on anything that talks over the internet: HTTP requests, AI models, app nodes. Most temporary failures disappear on the second try.

17.2 On Error

Also in a node's **Settings**, **On Error** decides what happens when the node fails after any retries:

CHOICE	RESULT
Stop Workflow (default)	The run fails; the error workflow fires
Continue	Carry on with the error details as the output
Continue (using error output)	The node gets a second, red **error** output you can connect to fallback steps

The error output is the most useful: for example, if enriching a lead fails, send it down the error output to a "needs manual review" sheet instead of losing it.

17.3 Error workflows

CREATE AN "ALERT ME" ERROR WORKFLOW

1. **Create a new workflow** called "Error handler".
2. **Add an Error Trigger** as the first node.
3. **Add a message node**, for example Gmail or Slack, with a message like:

```
TEXT

⚠️ {{ $json.workflow.name }} failed
Node: {{ $json.execution.lastNodeExecuted }}
Error: {{ $json.execution.error.message }}
Details: {{ $json.execution.url }}
```

4. **Save it.**
   An error workflow doesn't need publishing to work.
5. **Attach it.**
   In each important workflow, open the workflow menu (…) › **Settings**, choose "Error handler" under **Error Workflow**, and save.

The Error Trigger receives the failed workflow's name and ID, the execution ID and a link to it, the error message, and the last node that ran. If the failure happened in the trigger itself (for example a schedule couldn't activate), the data arrives under `trigger` instead, with less detail.

> **TIP**
> Make one "Alert me" error workflow, with Error Trigger → Gmail or Slack, and set it as the error workflow for everything you publish. You'll know about problems before anyone else does.

17.4 Stop and Error

Sometimes nothing technically fails, but the data is wrong: an order with a negative total, a form with no email. Add an If node to check, and on the bad path add **Stop and Error** with a clear message such as `Order {{ $json.id }} has no email`. The run fails cleanly and your error workflow tells you why.

17.5 Patterns for common problems

PROBLEM	PATTERN
A field is sometimes missing	Use `{{ $ifEmpty($json.phone, 'none') }}`, or an If node checking **is empty**
The API rate-limits you (error 429)	HTTP Request **Batching**, or Loop Over Items with a Wait node
A service is down for hours	Error output → store the item in a data table → a scheduled workflow retries later
A node returns no items and the flow stops	Turn on **Always Output Data** in node settings, then check for emptiness
Long runs time out	Split into sub-workflows; set **Timeout Workflow** deliberately

> **NOTE**
> When a node outputs zero items, the nodes after it don't run. That's usually what you want ("nothing new, nothing to do"), but it surprises people. **Always Output Data** makes the node output one empty item instead.

- [ ] Retry On Fail is on for my internet-facing nodes
- [ ] I've used an error output for at least one fallback path
- [ ] I have an "Error handler" workflow
- [ ] Every published workflow points at it
- [ ] I validate key data and fail clearly with Stop and Error

CHAPTER 18

Debugging Executions

When a workflow misbehaves, n8n has kept a detailed record of exactly what happened. Learning to read that record turns debugging from guesswork into a calm, methodical process.

IN THIS CHAPTER

- The Executions list and what to look for
- Reading a failed execution node by node
- Debug in editor and Copy to editor
- Retrying failed runs
- Custom execution data for searching
- A step-by-step debugging routine

18.1 The Executions list

Click the **Executions** tab of a workflow to see every saved run, or open **Overview › Executions** to see runs of all workflows you can access. Each row shows the status (**Succeeded**, **Error**, **Running**, **Waiting**, **Queued**), start time and duration. Filter by status or date to find the one you need.

> ⓘ NOTE
>
> Which runs appear depends on workflow settings (**Save failed / successful production executions**, **Save manual executions**) and on your plan's retention limits. On Cloud Starter, executions are kept for up to 7 days; on Pro, up to 30.

18.2 Reading a failed execution

Click a failed execution. The canvas opens read-only with the run's data. Nodes that ran show a green tick, and the failing node is red. Click any node to see its input and output exactly as they were. Open the red node to read the error message: it usually says what went wrong, and often which field.

18.3 Debug in editor

For a failed run, select **Debug in editor**; for a successful one, **Copy to editor**. n8n loads that run's data into your workflow and **pins** it on the first node. Now you can change the broken node, click **Execute workflow**, and re-run with the exact data that failed, without waiting for the real event to happen again. Remember to unpin afterwards.

> **NOTE**
> Debug in editor is available on all Cloud plans and on self-hosted Registered Community, Business and Enterprise. Self-hosted users get it by registering for free.

18.4 Retrying

From the Executions list you can **retry** a failed execution, either with the currently published workflow (after you've fixed it) or with the original version. Retries are a quick way to recover the items that failed during an outage.

18.5 Custom execution data

You can tag executions with your own searchable data, such as an order number, then filter the Executions list by it. Use the **Execution Data** node, or in a Code node:

JAVASCRIPT

```
$execution.customData.set('orderId', $json.id);
```

Custom execution data is available on Cloud Pro and Enterprise and self-hosted Registered Community and Enterprise.

18.6 A debugging routine

WHEN SOMETHING'S WRONG

1. **Find the run.**
   Open Executions and click the failed or suspicious run.
2. **Find the first surprise.**
   Walk through nodes left to right. Check the item count and values in each output. The bug is usually at the first node whose output isn't what you expected, not at the red one.
3. **Read the error carefully.**
   Copy it into a search of the community forum if it's unclear.
4. **Load the data.**
   Use **Debug in editor** so you can experiment with real input.
5. **Fix one thing, then test.**
   Execute the step, check output, repeat.
6. **Publish the fix**, then retry the failed executions if needed.

PROMPT

My n8n workflow fails at the node "[node name]" with this error:
[paste the error]. The node's input looks like this: [paste the input JSON].
The node is configured like this: [describe the settings]. Explain in plain
English what's wrong and how to fix it.

18.7 Decoding common error types

ERROR SAYS	USUALLY MEANS
401 Unauthorized	Credential wrong or expired: reconnect it
403 Forbidden	The account lacks permission for this action
404 Not Found	Wrong URL, ID or resource
429 Too Many Requests	Slow down: batching, Wait, Retry On Fail
500, 502, 503	The other service had a problem: retry later
"Referenced node is unexecuted"	An expression points at a node that didn't run on this path
"JSON parameter needs to be valid JSON"	A body or field contains broken JSON: check quotes and commas
Workflow timed out	Took longer than the allowed time: split it up

- [ ] I can find a run in the Executions list
- [ ] I can find the first node whose output looks wrong
- [ ] I've used Debug in editor to re-run with real data
- [ ] I know how to retry failed executions

CHAPTER 19

AI in n8n: The Basics

n8n has three layers of AI: AI *steps* inside workflows, AI *agents* that decide what to do, and AI that *builds workflows for you*. This chapter starts with the simplest and most reliable: using a language model as one step in an ordinary workflow.

IN THIS CHAPTER

- Language models, prompts and tokens in plain English
- Connecting a chat model: your own key or Gateway credits
- The ready-made AI nodes: chain, extractor, classifier, sentiment, summariser
- Getting structured answers you can use in later nodes
- n8n Assistant and other ways AI can build for you

19.1 A few AI words

WORD	MEANING
LLM (large language model)	The AI that reads and writes text, such as Claude, GPT or Gemini
Prompt	The instructions and text you send the model
System message	Standing instructions about role and style
Token	A chunk of text (about ¾ of a word); providers charge per token
Chat model	The n8n sub-node that connects to an LLM provider
Hallucination	When a model confidently states something false

19.2 AI steps in a workflow

Add an AI model node anywhere to summarize, classify, translate, or extract information. The **Basic LLM Chain**, **Information Extractor**, **Text Classifier**, and **Sentiment Analysis** nodes cover the common jobs. Connect a **chat model** (OpenAI, Anthropic Claude, Google Gemini, Mistral, Ollama for local models, and more) using your API key.

NODE	DOES	EXAMPLE
Basic LLM Chain	Sends a prompt, returns the answer	"Summarise this article in three bullets"
Information Extractor	Pulls named fields out of text	Invoice number, date and total from an email
Text Classifier	Sorts text into categories you define	Billing / technical / sales / spam
Sentiment Analysis	Judges tone	Positive, neutral or negative review
Summarization Chain	Summarises long documents in chunks	A 40-page PDF

> **TIP**
> On n8n Cloud Starter and Pro plans you can choose **Use Gateway credits** in a model's credential field. n8n handles billing, so you don't need your own OpenAI or Anthropic account. Great for getting started.

19.3 Connecting a chat model

SUMMARISE TEXT WITH CLAUDE

1. **Add a Basic LLM Chain** after a node that outputs text (say, an RSS item with `content`).
2. **Set the prompt.**
   Choose **Define below** and write: `Summarise this in two sentences for a busy reader: {{ $json.content }}`.
3. **Attach a chat model.**
   Click the **Model** connector under the node and pick **Anthropic Chat Model** (or OpenAI, Gemini…).
4. **Add a credential.**
   Paste an API key from the provider's console, or choose **Use Gateway credits** on Cloud.
5. **Pick a model** from the list, then **Execute step**. The answer appears in the output, ready for Slack or email.

> **NOTE**
> Gateway credits are available on Cloud Starter and Pro (not Enterprise or self-hosted). New accounts and trials include a small free amount. Top-ups expire 12 months after purchase. Rates per model are on n8n's service pricing page.

19.4 Getting structured answers

A paragraph of text is hard for later nodes to use. You want fields: `category`, `priority`, `reply`. Two ways:

- **Information Extractor**: list the attributes you want with a short description of each, or give a JSON example. The output has one field per attribute.
- **Require Specific Output Format** on a Basic LLM Chain or AI Agent, with a **Structured Output Parser** attached. Give it an example like the one below and n8n makes the model fill in exactly that shape.

EXAMPLE FOR THE STRUCTURED OUTPUT PARSER

```
{
  "category": "billing",
  "priority": "high",
  "summary": "Customer was charged twice in October",
  "suggested_reply": "Hi Sam, sorry about that..."
}
```

19.5 Writing good prompts

- [ ] Say who the AI is and who the output is for ("You are a support assistant replying to customers")
- [ ] Give the exact task and format ("Reply in under 80 words, plain text, no greeting line")
- [ ] Include the data clearly, between markers like `---`
- [ ] Give one example of a perfect answer
- [ ] Say what to do when unsure ("If the category is unclear, answer `other`")

19.6 Running models locally

Self-hosted users can run open models on their own hardware with **Ollama** and connect them with the **Ollama Chat Model** node. Your data never leaves your server, and there's no per-token bill, but you need a capable machine, and small models are less accurate than the big cloud ones.

19.7 Let AI build workflows for you

- **n8n Assistant** (in preview): a chat panel inside n8n that can plan, build, test and fix workflows from a description, and help with tasks like publishing. It's available on Cloud Starter and Pro and on self-hosted Community and Business. It uses **Assistant credits**: a monthly allowance comes with Cloud plans (1,600 a month on Starter). On self-hosted, you add your own model key.
- **From Claude or another MCP client**: connect to n8n's instance-level MCP server and say "build a workflow that saves Gmail attachments to Google Drive" (Chapter 22).

> ⚠ **WARNING**
> Review AI-built workflows before publishing, especially nodes that send messages, delete data, or spend money. Test with pinned or sample data first.

PROMPT

```
Build an n8n workflow that runs every weekday at 8am, reads new rows from
my Google Sheet "Leads" added in the last 24 hours, writes a two-line
summary of each with AI, and posts the summaries to the Slack channel
#sales. Use Gateway credits for the model. Don't publish it; I'll review it.
```

- [ ] I've connected a chat model with a key or Gateway credits
- [ ] I've used Basic LLM Chain for a summary
- [ ] I've extracted structured fields with Information Extractor or an output parser
- [ ] I review AI-built workflows before publishing

CHAPTER 20

The AI Agent Node in Depth

An AI step does one fixed job. An **AI agent** decides *what to do*: it reads a request, chooses which tools to use, uses them, looks at the results, and keeps going until it has an answer. This chapter explains how agents work in n8n and how to make them dependable.

IN THIS CHAPTER

- How an agent thinks: the reason–act loop
- The parts: model, memory, tools, system message
- Building a chatbot agent step by step
- Memory types and sessions
- Tools: app nodes, HTTP, sub-workflows, `$fromAI()`
- Human approval, guardrails and limits
- n8n's first-class Agents (preview)

20.1 How an agent works

An agent runs a loop. It **thinks** ("to answer this I need today's calendar"), **acts** (calls the calendar tool), **observes** the result, and repeats until it can reply. Each tool has a name and a description; the model reads those descriptions to decide what to use, so clear descriptions matter enormously.

20.2 The AI Agent node

The **AI Agent** node is a mini-agent inside a workflow. You attach:

PART	WHAT IT'S FOR
Chat model	The AI brain (e.g. Claude or GPT)
Memory	Remembers the conversation, e.g. **Simple Memory**, or a database for persistence
Tools	Things the agent can do: Gmail, Google Calendar, HTTP Request, a sub-workflow, or an **MCP Client Tool**
System message	Instructions: who it is and how to behave

The agent needs at least one tool. Its prompt comes either **from the previous node automatically** (it expects a field called `chatInput`, which the Chat Trigger provides) or from a prompt you **define below**.

BUILD A SIMPLE CHATBOT AGENT

1. **Add a Chat Trigger** as the first node.
2. **Add an AI Agent node** after it.
3. **Attach a chat model** (click **Chat Model +**) and choose, for example, **Anthropic Chat Model** with your credential or Gateway credits.
4. **Attach Simple Memory** so it remembers the conversation.
5. **Attach a tool**, for example **Google Calendar** with the operation *Get many events*, so it can answer "what's on my schedule?"
6. **Write the system message:**
   "You are my friendly scheduling assistant. Use the calendar tool to answer questions about my day. Be brief."
7. **Click Open chat** at the bottom of the canvas and talk to it.

20.3 Writing the system message

The system message is the agent's job description. A good one covers:

PROMPT

```
You are the support assistant for Bright Bikes, a bicycle shop.
Goal: answer customer questions about orders, opening hours and repairs.
Tools: use "Find order" to look up orders by email. Use "Knowledge base"
for policies. Never guess an order status; always look it up.
Style: friendly, plain English, under 100 words.
If you can't help, say so and offer to pass the message to a human.
Today's date is {{ $now.toFormat('d LLLL yyyy') }}.
```

> **TIP**
> Models don't know today's date. Adding it to the system message with an expression prevents a whole class of mistakes.

20.4 Agent options

OPTION	USE IT TO
Max Iterations	Cap how many think–act rounds it can take (default 10)
Return Intermediate Steps	Include the tool calls in the output, for checking its reasoning
Require Specific Output Format	Attach an output parser for structured results
Enable Streaming	Show the reply word by word in chat
Automatically Passthrough Binary Images	Let the model see images from earlier nodes

20.5 Memory

Without memory, every message is a fresh start. Memory sub-nodes store the conversation:

MEMORY NODE	STORES HISTORY	GOOD FOR
Simple Memory	Inside n8n	Getting started, single-server setups
Postgres Chat Memory	In a PostgreSQL database	Production, survives restarts
Redis Chat Memory	In Redis	Fast, production
MongoDB Chat Memory	In MongoDB	If you already use MongoDB

Two settings matter: the **Session Key**, which keeps different users' conversations apart (the Chat Trigger supplies a session ID automatically), and the **Context Window Length**, how many past exchanges to send back to the model. Longer memory means better context but more tokens.

> ⚠ **WARNING**
>
> Simple Memory doesn't work for published workflows when n8n runs in queue mode (Chapter 26). Use Postgres or Redis chat memory there.

20.6 Tools

Almost any app node can be attached as a tool: Gmail, Sheets, Notion, Slack, Calendar, HTTP Request and hundreds more. Other useful tools include **Call n8n Workflow Tool** (run a sub-workflow), **Code Tool**, **Calculator**, **Think** (gives the model room to plan), **Wikipedia**, **SerpAPI** or **SearXNG** for web search, **Vector Store Question Answer Tool** (Chapter 21), and **MCP Client Tool** (Chapter 22).

When you attach an app node as a tool, you choose which parameters you fix and which the AI fills in. For any field you want the model to decide, let the AI define it (the field shows an option for this); n8n writes it as a `$fromAI()` expression, which you can also type yourself:

```text
{{ $fromAI('recipient_email', 'The email address of the customer', 'string') }}
```

Fix everything you can (the sender, the channel, the sheet) and leave open only what the agent truly needs to choose. Fewer open fields means fewer mistakes.

> 💡 **TIP**
>
> Give each tool a clear name and description: "Find order: looks up a customer's orders by email address. Returns order number, status and date." The model chooses tools by reading these.

20.7 Human review for risky tools

You can make the agent ask a person before it uses certain tools. Click the agent's tool connector, find **Human review** in the tools panel, choose a channel (chat, Slack, Telegram and others), and connect the sensitive tools to it. When the agent wants to use one, the workflow pauses and sends an approval request; the action runs only if someone approves.

20.8 Guardrails

The **Guardrails** node checks text before it reaches your model, or before a model's answer reaches a customer. **Check Text for Violations** sends failing items to a **Fail** branch; **Sanitize Text** replaces sensitive content instead. Available checks include keywords, jailbreak attempts, NSFW content, personal data (PII) such as card or phone numbers, secret keys, staying on topic, URLs, and your own custom or regex rules. LLM-based checks need a chat model attached.

20.9 Keeping agents dependable

- [ ] One clear job per agent; split big jobs into several agents or sub-workflows
- [ ] Fixed parameters wherever possible, `$fromAI()` only where needed
- [ ] Human review on anything that sends, deletes or pays
- [ ] Guardrails on public-facing input and output
- [ ] Max Iterations set to a sensible number
- [ ] Tested with tricky questions before publishing

20.10 Agents (first-class, in preview)

Newer versions of n8n also offer **Agents** as their own kind of project item, alongside workflows. You configure a model, instructions, tools, skills, and a knowledge base in the **Agent Builder**, then let people reach the agent through chat, channels, or schedules. Agents can call your workflows as tools. The feature is in preview, so expect changes. Owners can switch it on or off in Settings › Agents.

CHOOSE	WHEN
AI Agent node in a workflow	The flow is mostly fixed, with AI as one step
A first-class Agent	The AI owns the conversation and decides what to do

CHAPTER 21

Knowledge for Your AI: Vector Stores and RAG

A language model knows a lot about the world, but nothing about *your* price list, handbook or support articles. **RAG** (retrieval-augmented generation) fixes that: before answering, the AI looks up the most relevant passages from your documents and uses them. This chapter explains the idea and builds a working knowledge base.

IN THIS CHAPTER

- Embeddings and vector stores, without the maths
- The two halves of RAG: loading and asking
- Building a knowledge base from documents
- Connecting it to an agent as a tool
- Choosing a vector store for production
- Making answers more accurate

21.1 The idea in plain English

Imagine a librarian who has read every page of your documents and sorted them by *meaning*. When a question arrives, they pull out the three most relevant pages and hand them to the writer, who answers using only those pages.

- An **embedding** is a list of numbers that captures the meaning of a piece of text. Texts with similar meanings get similar numbers. An **embeddings model** (for example OpenAI, Gemini, Cohere, Mistral or Ollama embeddings) creates them.
- A **vector store** is a database that stores those numbers alongside the text, and can quickly find the passages closest in meaning to a question.
- **Chunking** means cutting long documents into passages (a few paragraphs each) so each one can be found separately. A **text splitter** does this.

21.2 The two halves of RAG

HALF	RUNS	STEPS
Loading (ingestion)	When documents change	Get files → split into chunks → embed → store
Asking (retrieval)	Every question	Embed the question → find similar chunks → answer with them

21.3 Build a knowledge base

LOAD DOCUMENTS INTO A VECTOR STORE

1. **Start with a trigger and your documents.**
   For a first test, use a Manual Trigger and a Google Drive node that downloads a few PDFs or Google Docs.

2. **Add a vector store node** and choose the **Insert Documents** operation. To begin, use **Simple Vector Store** and give it a **Memory Key** such as `handbook`.

3. **Attach an embeddings model** to its **Embedding** connector, for example **Embeddings OpenAI**.

4. **Attach a document loader**
   Default Data Loader, set to binary (for files) or JSON (for text fields).

5. **Attach a text splitter** to the loader: **Recursive Character Text Splitter** is a good default. Try a chunk size of about 1,000 characters with 200 characters of overlap.

6. **Execute the workflow.**
   The output shows how many chunks were stored.

21.4 Ask questions with an agent

CONNECT THE KNOWLEDGE BASE TO A CHATBOT

1. **Build the chatbot** from Chapter 20: Chat Trigger → AI Agent with a chat model and memory.
2. **Add a vector store as a tool.**
   On the agent's **Tool** connector, add the same vector store node with the operation **Retrieve Documents (As Tool for AI Agent)**, and the same Memory Key, `handbook`.
3. **Describe the tool.**
   Name it `company_handbook` and describe it: "Searches the staff handbook. Use it for any question about policies, holidays or expenses."
4. **Use the same embeddings model** as when loading. Different models produce incompatible numbers.
5. **Set a Limit** of 4 to 6 chunks, so the model gets enough context without drowning.
6. **Tell the agent to rely on it.**
   Add to the system message: "Answer only from the handbook tool. If it doesn't contain the answer, say you don't know."

21.5 Choosing a vector store

> ⚠ WARNING
>
> **Simple Vector Store** keeps data in n8n's memory. It's lost when n8n restarts, and every user on the instance can access it. Use it for experiments only.

VECTOR STORE	GOOD FOR
PGVector	You already run PostgreSQL (add the pgvector extension)
Supabase	Hosted Postgres with a free tier
Qdrant	Fast, open source, self-hostable or cloud
Pinecone	Fully managed cloud service
Others	Chroma, Milvus, MongoDB Atlas, Redis, Weaviate, Zep, Azure AI Search

The workflow shape is the same whichever you pick: only the node and credential change.

21.6 Keeping it fresh

Run the loading workflow whenever documents change: on a schedule, or with a Google Drive trigger on file updates. To avoid duplicates, clear the old chunks for a document before inserting new ones (Simple Vector Store has a **Clear Store** option; other stores let you delete by metadata or use a separate collection per source).

21.7 Improving answers

PROBLEM	TRY
Answers miss obvious facts	Smaller chunks, more results (higher Limit)
Answers mix up topics	Larger chunks with overlap; clearer tool description
Agent ignores the knowledge base	Say in the system message to always search it first
Made-up answers	“If it isn’t in the documents, say you don’t know”
Slow or expensive	Fewer chunks; a cheaper model for simple questions
Ranking is poor	Add a reranker such as **Reranker Cohere**

- [ ] I can explain embeddings and vector stores in my own words
- [ ] I’ve loaded documents with an embeddings model, loader and splitter
- [ ] My agent uses the vector store as a tool with a clear description
- [ ] I use the same embeddings model for loading and asking
- [ ] I’ve picked a persistent vector store for production

CHAPTER 22

MCP: n8n and Other AI Apps

MCP (Model Context Protocol) is the standard way for AI apps to use outside tools, like a universal plug socket. n8n speaks MCP in both directions: your agents can use other MCP servers, and AI apps such as Claude can use n8n. See the *MCP* manual in this series for the bigger picture.

IN THIS CHAPTER

- The three ways n8n uses MCP
- Giving your agent tools from MCP servers
- Turning a workflow into an MCP server
- Letting Claude or ChatGPT search, run and build your workflows

FEATURE	DIRECTION	USE IT TO
MCP Client Tool node	n8n uses other MCP servers	Give your AI Agent tools from GitHub, Notion, Linear, and more. The node panel also has a one-click **MCP Servers** registry
MCP Server Trigger node	Other AI apps use one n8n workflow	Turn a workflow into a custom MCP server that Claude or Cursor can call
Instance-level MCP server	Other AI apps use all of n8n	Let Claude, ChatGPT, or a coding agent search, run, *and build* your workflows from a chat

22.1 Using MCP servers in your agents

ADD A REGISTRY MCP SERVER IN ONE CLICK

1. **Open your AI Agent's tools panel** by clicking **Tool +**.
2. **Search for the service** (for example Notion or Linear) or open the **MCP Servers** section.
3. **Select it and sign in** when asked. n8n creates the MCP credential for you.

4. **Choose Tools to include**
   all, a selected list, or all except some.

If a server isn't in the registry, add an **MCP Client Tool** and enter its **endpoint URL** (for example `https://mcp.notion.com/mcp`) and authentication (bearer token, header, multiple headers or OAuth2). To use an MCP tool as an ordinary workflow step rather than through an agent, use the **MCP Client** node and pick the tool from its list.

USE A BUILT-IN TOOL WHEN…	USE AN MCP SERVER WHEN…
You know the exact action needed	The agent needs a range of actions
You want to fix most parameters	You're happy for it to choose tools and arguments

22.2 Turn a workflow into an MCP server

EXPOSE TOOLS WITH THE MCP SERVER TRIGGER

1. **Create a workflow** with an **MCP Server Trigger**.
2. **Attach tools** to it: app nodes, or **Call n8n Workflow Tool** nodes that run your own workflows.
3. **Set Authentication** to **Bearer auth** or **Header auth**, and choose a memorable **Path**.
4. **Publish**, then copy the **Production URL** from the top of the node.
5. **Add it to your AI app** as a remote MCP server, with the token.

For apps that only support local (stdio) servers, such as some Claude Desktop setups, n8n's docs show bridging with `mcp-remote`:

CLAUDE_DESKTOP_CONFIG.JSON

```
{
  "mcpServers": {
    "n8n": {
      "command": "npx",
      "args": ["mcp-remote", "<MCP_URL>",
               "--header", "Authorization: Bearer ${AUTH_TOKEN}"],
      "env": { "AUTH_TOKEN": "<MCP_BEARER_TOKEN>" }
    }
  }
}
```

22.3 Instance-level MCP: let AI build for you

The instance-level server connects an AI app to your whole n8n. From a chat, the AI can search workflows, run the ones you've enabled, create and edit workflows (n8n 2.13 onward), and manage data tables.

CONNECT CLAUDE TO YOUR N8N

1. **Enable it.**
   As owner or admin, open `Settings` › `Instance-level MCP` and select **Enable MCP access**.
2. **Connect a client.**
   In **Connection details**, select **Connect**. Choose **OAuth (recommended)** or an API key. Web clients offer one-click setup; otherwise copy the **Server URL**, which ends in `/mcp-server/http`.
3. **Approve access** when the client sends you back to n8n.
4. **Enable workflows.**
   Clients can only see previews until you enable a workflow: open it, go to the workflow menu › **Settings** and toggle **Available in MCP**, or use **Workflows enabled** on the MCP settings page.

> ☆ IMPORTANT
> Every connected client can see every workflow you've enabled for MCP. Enable only what you're happy for AI apps to run. You can revoke a client at any time under **Connected clients**.

> ⚠ WARNING
> Review AI-built workflows before publishing, especially nodes that send messages, delete data, or spend money. Test with pinned or sample data first.

PROMPT

```
Using my n8n MCP connection, build a workflow that triggers when a file is
added to my Google Drive folder "Receipts", extracts the vendor, date and
total with AI, and appends them to the Google Sheet "Expenses 2026".
Test it with a sample, show me the result, and don't publish it.
```

- [ ] I've given an agent tools from an MCP server
- [ ] I've exposed one workflow with the MCP Server Trigger and authentication
- [ ] I've connected an AI app to instance-level MCP

CHAPTER 23

Recipes: Everyday Automations

The next three chapters are a cookbook: twelve complete workflows you can build in an afternoon each. Every recipe lists what you need, the steps, and ideas to extend it. Build them as written first, then make them yours. Exact field names in app nodes can vary slightly between versions; pick the closest match.

IN THIS CHAPTER

- Recipe 1: RSS-to-Slack daily digest
- Recipe 2: a daily report email
- Recipe 3: nightly file backup
- Recipe 4: social post scheduler

TIP

For every recipe: test each node as you add it, pin data on slow or expensive steps, attach your error workflow (Chapter 17), and only then publish.

23.1 Recipe 1: RSS-to-Slack digest

Goal: every morning, post a short AI-written digest of new articles from your favourite sites to a Slack channel.

You need: Slack credential, a chat model (or Gateway credits). **Nodes:** Schedule Trigger, RSS Read, Remove Duplicates, Limit, Aggregate, Basic LLM Chain, Slack.

BUILD THE DIGEST

1. **Schedule Trigger:**
   Days, at 8am.

2. **RSS Read:**
   paste a feed URL (most blogs and news sites have one; try adding `/feed` or `/rss` to the site address). For several feeds, put the URLs in an Edit Fields node as a list, Split Out, and feed each into RSS Read using `{{ $json.url }}`.

3. **Remove Duplicates:**
   operation **Remove Items Processed in Previous Executions**, deduping on `{{ $json.link }}`. Now you only see new articles.

4. **Limit:**
   keep 10 items so the digest stays short.

5. **Aggregate:**
   combine the `title`, `link` and `contentSnippet` fields into lists, making one item.

6. **Basic LLM Chain:**
   prompt: `Write a friendly Slack digest. For each article give the title as a link and one sentence on why it matters. Articles: {{ JSON.stringify($json) }}`.

7. **Slack:**
   send a message to `#reading` with the chain's `text` output.

Extend it: add a Text Classifier to tag articles by topic; post only on weekdays; send to email or Telegram instead.

23.2 Recipe 2: daily report email

Goal: at 6pm, email yourself a summary of today's sales from a Google Sheet.

Nodes: Schedule Trigger, Google Sheets, Filter, Summarize, Code (or Edit Fields), Gmail.

BUILD THE REPORT

1. **Schedule Trigger:**
   Days, at 6pm.

2. **Google Sheets:**
   operation to get rows from your "Sales" sheet.

3. **Filter:**
keep rows where `{{ $json.date }}` equals `{{ $today.toFormat('yyyy-MM-dd') }}` (match your sheet's date format).

4. **Summarize:**
sum `amount`, count rows, grouped by `product` if you like.

5. **Code:**
build an HTML table with recipe 5 from Chapter 10, adding a total line.

6. **Gmail:**
send to yourself, subject `Sales {{ $today.toFormat('d LLL') }}`, email type HTML, message `{{ $json.html }}`.

> **TIP**
> If no sales happened, the Filter outputs nothing and the email isn't sent. Prefer a "No sales today" email? Turn on **Always Output Data** on the Filter and add an If node checking for an empty result.

23.3 Recipe 3: nightly file backup

Goal: every night, copy new files from one cloud folder to another (for example Google Drive to Dropbox), and log what was copied.

Nodes: Schedule Trigger, Google Drive (search/list files), Data Table, Google Drive (download), Dropbox (upload), Data Table (insert).

BUILD THE BACKUP

1. **Schedule Trigger:**
Days, at 2am.

2. **Google Drive:**
list or search the files in your chosen folder.

3. **Data Table › If Row Does Not Exist:**
check each file ID against a `backed_up` table (Chapter 15), so only new files continue.

4. **Google Drive › Download:**
download each file by ID. It arrives as binary data.

5. **Dropbox › Upload:**
set the path to `/Backups/{{ $json.name }}` and the binary property to the download's slot (usually `data`).

6. **Data Table › Insert:**
record the file ID, name and `{{ $now.toISO() }}`.

Self-hosted variation: use **Read/Write Files from Disk** to save files to a folder on your server that your system backups already cover. Large files use memory, so add Loop Over Items with a small batch size if you back up many big files.

23.4 Recipe 4: social post scheduler

Goal: write posts in a spreadsheet with a date and time; n8n publishes each one when it's due.

Sheet columns: `post_text`, `image_url`, `publish_at` (e.g. `2026-10-20 09:00`), `status`.

Nodes: Schedule Trigger, Google Sheets, Filter, social node (LinkedIn, X, Facebook…), Google Sheets update.

BUILD THE SCHEDULER

1. **Schedule Trigger:**
every 15 minutes.
2. **Google Sheets:**
get rows from the "Posts" sheet.
3. **Filter:**
`status` **is empty** AND `{{ $json.publish_at.toDateTime() }}` **is before** `{{ $now }}` (Date & Time comparison).
4. **Social node:**
for example **LinkedIn › Create a post** with `{{ $json.post_text }}`. If there's an image, download it first with HTTP Request and attach the binary.
5. **Google Sheets › Update row:**
set `status` to `posted` and add the time, matching on the row number.

> ⚠ WARNING
> Always mark rows as posted *after* posting succeeds, and attach your error workflow. Otherwise a failure could cause a post to go out twice, or not at all.

Extend it: add a Basic LLM Chain that turns one long post into short versions for each network; add an approval step with a Wait node set to **On Form Submitted**.

- [ ] I've built at least one recipe end to end
- [ ] It uses Remove Duplicates or a data table so it never repeats work

CHAPTER 24

Recipes: Business Workflows

These recipes handle the admin that eats into a small business's week: capturing leads, enriching them, processing invoices, keeping tools in sync and watching the web for changes.

IN THIS CHAPTER

- Recipe 5: form to CRM
- Recipe 6: lead enrichment
- Recipe 7: invoice processing with AI
- Recipe 8: Notion sync
- Recipe 9: web page change monitor

24.1 Recipe 5: form to CRM

Goal: a contact form that adds the person to your CRM, emails them a welcome, and pings you.

Nodes: n8n Form Trigger, Edit Fields, CRM node (HubSpot, Pipedrive, Airtable or a Google Sheet), Gmail, Slack or Telegram.

BUILD IT

1. **n8n Form Trigger:**
   title "Get in touch"; fields Name (Text, required), Email (Email, required), Company (Text), Message (Textarea). **Respond When:** Form Is Submitted.

2. **Edit Fields:**
   tidy the data: `name` = `{{ $json.Name.trim().toTitleCase() }}`, `email` = `{{ $json.Email.trim().toLowerCase() }}`.

3. **CRM node:**
   choose the create-or-update contact operation (for example HubSpot **Contact › Create or Update**), mapping email, name and company. “Create or update” avoids duplicates when someone submits twice.
4. **Gmail:**
   send a short welcome to `{{ $json.email }}`.
5. **Slack:**
   post `New lead: {{ $json.name }} ({{ $json.company }})` to `#sales`.

> **TIP**
> Spam filler? Add a **Hidden Field** to the form and drop any submission where it’s filled in (bots fill everything), or run the message through a Text Classifier and discard `spam`.

24.2 Recipe 6: lead enrichment

Goal: when a new lead arrives, look up their company’s website, summarise what it does, and score the lead.

Nodes: a trigger (Recipe 5’s form, a CRM trigger, or a Sheets trigger), HTTP Request, HTML, Information Extractor, CRM update.

BUILD IT

1. **Get the domain:**
   `{{ $json.email.extractDomain() }}`. Skip free mail domains with a Filter (`gmail.com`, `outlook.com`…).
2. **HTTP Request:**
   GET `https://{{ $json.domain }}`, response format **Text**. Turn on **Never Error** and Retry On Fail: some sites are slow.
3. **HTML › Extract HTML Content:**
   pull the `title`, the meta description and the `body` text.
4. **Information Extractor:**
   attributes `industry`, `company_size_guess`, `one_line_summary`, `fit_score` (1–5, with your ideal-customer description in the system prompt).
5. **CRM update:**
   write the fields back to the contact; if `fit_score` ≥ 4, also alert sales.

Extend it: use a dedicated enrichment API through HTTP Request; route low-fit leads to a nurture email sequence.

24.3 Recipe 7: invoice processing with AI

Goal: invoices emailed to `invoices@` are read automatically, logged in a sheet, and filed in Drive.

Nodes: Gmail Trigger (or IMAP Email trigger), Filter, Extract From File, Information Extractor, Google Drive, Google Sheets, If.

BUILD IT

1. **Gmail Trigger:**
   poll every few minutes for messages to your invoices address or with an "Invoices" label, downloading attachments.
2. **Filter:**
   keep items whose attachment is a PDF (`mimeType` is `application/pdf`).
3. **Extract From File › Extract From PDF:**
   point it at the attachment's binary property. Out comes the invoice's text.
4. **Information Extractor:**
   from the text, extract `supplier`, `invoice_number`, `invoice_date`, `due_date`, `currency`, `total` (number).
5. **Google Drive › Upload:**
   save the PDF into "Invoices/2026" named `{{ $json.supplier }} {{ $json.invoice_number }}.pdf`.
6. **Google Sheets › Append Row:**
   log the fields with the Drive link.
7. **If:**
   total above your approval limit → Slack your manager with the details.

> **DANGER**
> AI extraction is very good but not perfect. Keep a person in the loop for payments: have the workflow *log and alert*, not pay. Add a "checked" column you tick by hand.

> **TIP**
> Scanned paper invoices have no text layer, so Extract From PDF returns nothing. Send the file to a vision-capable model or an OCR service instead.

24.4 Recipe 8: Notion sync

Goal: keep a Notion database of tasks in step with a Google Sheet (or another tool), so teammates can work wherever they like.

Nodes: Schedule Trigger, Notion (get database pages), Google Sheets (get rows), Compare Datasets, Notion create/update, Sheets update.

BUILD A ONE-WAY SYNC (SHEET → NOTION)

1. **Connect Notion.**
   Create an internal integration in Notion's developer settings, copy its secret into an n8n Notion credential, and **share your database with the integration** from the database's connections menu.
2. **Schedule Trigger:**
   every 30 minutes.
3. **Google Sheets:**
   get all rows (each needs a stable `task_id`).
4. **Notion › Database Page › Get Many:**
   get all pages from the database, with their `task_id` property.
5. **Compare Datasets:**
   compare on `task_id`. Its outputs separate items only in the sheet (new), in both but different (changed), and only in Notion.
6. **Notion › Create:**
   for new items. **Notion › Update:** for changed items, using the page ID from the Notion side.

> NOTE
> Two-way sync is much harder: you need to know which side changed most recently, and avoid ping-ponging. Start one-way; add the other direction only if you truly need it, using a "last edited" timestamp on both sides.

24.5 Recipe 9: web page change monitor

Goal: check a page (a price, a job board, a "sold out" notice) every hour and alert you when it changes.

Nodes: Schedule Trigger, HTTP Request, HTML, Data Table, If, Telegram or email.

BUILD IT

1. **Schedule Trigger:**
   every hour.
2. **HTTP Request:**
   GET the page, response format **Text**.
3. **HTML › Extract HTML Content:**
   extract just the part you care about with a **CSS selector** (right-click the element in your browser, **Inspect**, then copy the selector), into a field `value`.
4. **Data Table › Get:**
   read the last saved value for this URL.
5. **If:**
   new `value` is not equal to the saved one.
6. **True branch:**
   send `Changed: {{ $json.value }}` by Telegram, then **Data Table › Upsert** the new value.

> ⚠ WARNING
> Respect websites: check politely (hourly, not every minute), follow their terms, and prefer an official API or RSS feed if one exists. Pages that build their content with JavaScript may need a scraping service rather than a plain HTTP request.

- [ ] Lead data is cleaned before it reaches my CRM
- [ ] AI-extracted invoice data is checked by a person before payment
- [ ] My Notion integration has access to the database
- [ ] My monitor checks politely and stores the last value

CHAPTER 25

Recipes: AI Workflows

The last three recipes put AI to work: sorting your inbox, answering customers from your own documents, and a personal assistant you can message from your phone.

IN THIS CHAPTER

- Recipe 10: email triage with AI
- Recipe 11: AI support chatbot with RAG
- Recipe 12: Telegram assistant bot

25.1 Recipe 10: email triage with AI

Goal: every new email is classified, labelled, and, for customer questions, given a draft reply waiting for you to check.

Nodes: Gmail Trigger, Text Classifier, Gmail (add label), Basic LLM Chain with Structured Output Parser, Gmail (create draft), Slack.

BUILD IT

1. **Gmail Trigger:**
   event **Message Received**, polling every minute or five. Keep **Simplify** on.
2. **Text Classifier:**
   input `Subject: {{ $json.Subject }} Body: {{ $json.snippet }}`. Categories with short descriptions: `customer_question`, `invoice`, `newsletter`, `personal`, `spam`. Each category becomes an output.
3. **Gmail › Add Label:**
   on each output, add the matching label to `{{ $json.id }}`.

4. **On customer_question:**
   a **Basic LLM Chain** with your tone and policies in the prompt, and a Structured Output Parser returning `urgency` (low/medium/high) and `reply`.
5. **Gmail › Create draft:**
   a reply to the sender with the AI's `reply`. You review and send it yourself.
6. **If urgency is high:**
   Slack yourself a link to the email.

> **TIP**
> Start with drafts, never automatic sending. After a few weeks of reading the drafts, you'll know which categories are safe to automate further.

> **NOTE**
> Every new email is one execution. A busy inbox can use thousands a month. Narrow the trigger with Gmail search filters (for example only the inbox, not promotions) to save executions and AI costs.

25.2 Recipe 11: AI support chatbot with RAG

Goal: a chat window, on its own page or your website, that answers customer questions from your help articles and hands over to a human when stuck.

Two workflows: a loader (documents → vector store) and the chatbot.

WORKFLOW A: LOAD YOUR HELP ARTICLES

1. **Trigger:**
   Schedule (daily) or a Google Drive trigger on your "Help articles" folder.
2. **Google Drive:**
   download the documents (or HTTP Request to fetch your help-centre pages).
3. **Vector store › Insert Documents:**
   PGVector, Supabase, Qdrant or Pinecone, with an embeddings model, Default Data Loader and Recursive Character Text Splitter (Chapter 21).

WORKFLOW B: THE CHATBOT

1. **Chat Trigger:**
   Make Chat Publicly Available on, mode **Hosted Chat** (or **Embedded Chat** for your site). Set a title and welcome message in the options.

2. **Guardrails (optional but wise):**
   Check Text for Violations with jailbreak and topical alignment checks; the Fail branch replies politely that you can only help with your products.

3. **AI Agent:**
   chat model, **Postgres Chat Memory** (or Simple Memory to start), and the vector store as a tool (**Retrieve Documents (As Tool for AI Agent)**) named `help_articles`.

4. **Add a hand-over tool:**
   a **Call n8n Workflow Tool** that runs a sub-workflow emailing your support inbox with the conversation and the customer's address. Describe it: "Use when you can't answer from help_articles, or the customer asks for a person."

5. **System message:**
   your shop's name, "answer only from help_articles," tone, length, and "never promise refunds; hand over instead."

6. **Test hard.**
   Ask easy questions, off-topic questions, and tricky ones. Read **Return Intermediate Steps** to see which tools it used.

7. **Publish** and share the chat URL, or embed it on your site.

> ☆ IMPORTANT
> Tell customers they're talking to an AI, and give them an easy path to a human. Check your local rules on data protection before storing chat histories.

25.3 Recipe 12: Telegram assistant bot

Goal: message a private Telegram bot from your phone: "what's on tomorrow?", "add milk to my list", "summarise this link."

Nodes: Telegram Trigger, If (security check), AI Agent with memory and tools, Telegram (send message).

BUILD IT

1. **Create the bot.**
   In Telegram, message **@BotFather**, send `/newbot`, follow the prompts, and copy the **bot token** into an n8n Telegram credential.

2. **Telegram Trigger:**
   updates **Message**. Send your bot a message while listening to see the data, and note your own `chat.id`.

3. **If:**
   continue only when `{{ $json.message.chat.id }}` equals your ID, so strangers can't use your assistant.

4. **AI Agent:**
   prompt `{{ $json.message.text }}`; chat model; Simple Memory with session key `{{ $json.message.chat.id }}`; tools such as Google Calendar (get events), Google Tasks or a data table (your list), and HTTP Request (fetch a link).

5. **Telegram › Send Message:**
   chat ID `{{ $('Telegram Trigger').item.json.message.chat.id }}`, text `{{ $json.output }}`.

6. **Publish.**
   Telegram needs a public HTTPS address, so use Cloud or your server from Chapter 5.

> ⚠ **WARNING**
> Telegram allows only one webhook per bot. Testing in the editor while the workflow is published redirects messages to the test URL. Publish again after testing, or use a second bot for development.

Extend it: send voice notes and transcribe them with a speech-to-text model; add a daily morning message from a Schedule Trigger.

- [ ] My email triage creates drafts, not automatic replies
- [ ] My chatbot answers from my documents and can hand over to a human
- [ ] My Telegram bot only answers my own chat ID
- [ ] Each AI workflow has guardrails or approvals where it matters

CHAPTER 26

Running Your Own n8n: Operations

If you self-host, you are now the person who keeps n8n healthy. The good news: with Docker Compose, the routine is short. This chapter covers backups, updates, configuration, scaling with queue mode, and security hardening.

IN THIS CHAPTER

- What to back up, and how to restore
- Updating safely
- Environment variables you'll actually use
- Keeping the database small
- Queue mode with Redis and workers
- A security hardening checklist
- Useful command-line tools

If you self-host for real (not just to experiment), cover these basics:

TOPIC	WHAT TO DO
Always-on hosting	Use a small cloud server (DigitalOcean, Hetzner, AWS…) or a service with one-click n8n, rather than your laptop
Docker Compose	n8n's recommended production setup, with a proper database (PostgreSQL)
HTTPS and a domain	Put n8n behind a reverse proxy with a certificate, and set `N8N_WEBHOOK_URL` to your public address so webhooks work
Encryption key	n8n encrypts credentials with a key in its data folder. Back it up; without it, saved credentials can't be decrypted
Backups	Back up the data volume and database regularly; export important workflows too
Updates	Update regularly for features and security fixes; read the release notes for breaking changes first
Access	Use a strong owner password and two-factor authentication; don't expose the editor to the internet unprotected

26.1 Backups

A complete backup has two parts:

1. **The `.n8n` folder** (in Docker, the `n8n_data` volume at `/home/node/.n8n`). It holds the `config` file with the encryption key and, with SQLite, the whole database.
2. **Your PostgreSQL database**, if you use one, backed up with Postgres's own tools.

Also keep a copy of your `.env` file and Compose file. Here's a simple nightly backup for the Chapter 5 setup, run from `~/n8n-compose`:

BACKUP.SH

```
#!/bin/sh
cd ~/n8n-compose
DAY=$(date +%F)
mkdir -p backups

# 1. Database dump
docker compose exec -T postgres \
  pg_dump -U n8n_admin n8n > backups/db-$DAY.sql

# 2. The .n8n folder (encryption key, settings)
docker run --rm -v n8n-compose_n8n_data:/data -v "$PWD/backups":/b \
  alpine tar czf /b/n8n-data-$DAY.tgz -C /data .

# 3. Keep 14 days
find backups -type f -mtime +14 -delete
```

Make it executable with `chmod +x backup.sh`, and schedule it with `crontab -e`, adding a line like `30 3 * * * /root/n8n-compose/backup.sh`. Docker names volumes after the project folder, so check yours with `docker volume ls`.

> **DANGER**
> A backup on the same server dies with the server. Copy the `backups` folder elsewhere regularly (another provider's storage, or your own computer), and test a restore at least once.

Exporting workflows and credentials

n8n's command line can export everything as readable JSON. In Docker, run commands with `docker compose exec -u node n8n`:

TERMINAL

```
n8n export:workflow --backup --output=backups/workflows/
n8n export:credentials --backup --output=backups/credentials/
```

To restore into a fresh instance, complete the owner setup first, then:

TERMINAL

```
n8n import:workflow --separate --input=backups/workflows/
n8n import:credentials --separate --input=backups/credentials/
```

Imported workflows start unpublished. Credentials only decrypt if the new instance uses the **same encryption key**. CLI exports don't include users, execution history, variables or instance settings, so keep the full backup too.

26.2 Updating

n8n releases a new minor version most weeks. Update at least monthly so you never have to jump many versions at once.

UPDATE THE CHAPTER 5 SETUP

1. **Read the release notes** for anything marked as a breaking change.
2. **Back up** (run `backup.sh`).
3. **Change the version.**
   Edit `N8N_VERSION` in `.env` to the new stable version.

4 **Pull and restart:**

```
TERMINAL
docker compose pull
docker compose up -d
```

5 **Check it.**
`docker compose logs --tail=50 n8n`, then open the editor and look at a few recent executions.

To roll back, set the old version again and restart. If the update migrated the database, restore your database backup too.

26.3 Environment variables you'll actually use

VARIABLE	DEFAULT	WHAT IT DOES
`N8N_WEBHOOK_URL`	–	Public base URL for webhooks (replaces `WEBHOOK_URL`)
`N8N_PROXY_HOPS`	`0`	Number of reverse proxies in front of n8n
`N8N_HOST` / `N8N_PROTOCOL`	`localhost` / `http`	Public host name and protocol
`N8N_EDITOR_BASE_URL`	–	Public URL of the editor, used in emails
`N8N_ENCRYPTION_KEY`	Random	Key that encrypts credentials
`GENERIC_TIMEZONE`	New York	Default time zone for schedules
`EXECUTIONS_DATA_PRUNE`	`true`	Delete old execution data automatically
`EXECUTIONS_DATA_MAX_AGE`	`336`	Hours to keep executions (14 days)
`EXECUTIONS_DATA_PRUNE_MAX_COUNT`	`10000`	Maximum executions kept
`EXECUTIONS_TIMEOUT`	`-1`	Default workflow timeout in seconds (off)
`N8N_CONCURRENCY_PRODUCTION_LIMIT`	`-1`	Cap on simultaneous production runs
`N8N_PAYLOAD_SIZE_MAX`	`16`	Max request size in MiB
`N8N_DATA_TABLES_MAX_SIZE_BYTES`	200 MiB	Data table storage limit
`N8N_BLOCK_ENV_ACCESS_IN_NODE`	`false`	Stop workflows reading environment variables
`N8N_RESTRICT_FILE_ACCESS_TO`	–	Folders file nodes may use
`NODES_EXCLUDE`	–	Nodes to disable entirely
`N8N_PUBLIC_API_DISABLED`	`false`	Turn off n8n's REST API
`N8N_DIAGNOSTICS_ENABLED`	`true`	Anonymous telemetry to n8n

Add variables to the `environment:` section of the n8n service (or to `.env` and reference them), then run `docker compose up -d`.

26.4 Keeping the database small

Every execution stores its data. Busy instances fill disks quickly. By default n8n prunes executions older than 14 days or beyond 10,000. To save more space, set workflows to not save successful production executions (workflow settings), keep failures for debugging, and lower `EXECUTIONS_DATA_MAX_AGE`.

26.5 Scaling with queue mode

One n8n process handles a lot, but very busy instances can use **queue mode**: the main instance receives triggers and webhooks and puts jobs in a queue; separate **worker** processes run them.

PIECE	ROLE
Main	Editor, triggers, webhooks; creates jobs
Redis	The queue (message broker)
Workers	Run executions; add more to scale
PostgreSQL	Required; SQLite isn't supported in queue mode
Webhook processors (optional)	Extra processes that receive webhook traffic

The essentials:

- Set `EXECUTIONS_MODE=queue` on main and every worker, and point them at Redis with `QUEUE_BULL_REDIS_HOST` and `QUEUE_BULL_REDIS_PORT` (plus `QUEUE_BULL_REDIS_PASSWORD` if set).
- Give **every** worker the same `N8N_ENCRYPTION_KEY` and database settings as main.
- Start a worker with the `worker` command, for example a second service using the n8n image with `command: worker`, or `n8n worker --concurrency=5`. Worker concurrency defaults to 10; n8n recommends 5 or more.
- Simple Memory and binary data in filesystem mode don't work in queue mode; use database chat memory and S3 storage instead.

> **NOTE**
> Queue mode itself is free to use. Running several *main* instances (multi-main) and the workers view in settings need paid licences.

26.6 Security hardening checklist

- [ ] Owner and all users have strong passwords and 2FA (`N8N_MFA_ENFORCED_ENABLED=true` can enforce it)

- [ ] Only ports 22, 80 and 443 are open; n8n's port 5678 isn't exposed
- [ ] HTTPS everywhere, through Caddy, Traefik or a load balancer
- [ ] n8n and the server's operating system are updated monthly
- [ ] Backups run nightly, are copied off the server, and a restore has been tested
- [ ] Public webhooks, forms and chats use authentication where possible
- [ ] `N8N_BLOCK_ENV_ACCESS_IN_NODE=true` if others build workflows on your instance
- [ ] Risky nodes (Execute Command, SSH, file access) are excluded with `NODES_EXCLUDE` if unneeded
- [ ] Community nodes only from authors you trust
- [ ] `n8n audit` run occasionally to spot risky settings

26.7 Command-line toolbox

Run these inside the container with `docker compose exec -u node n8n <command>`:

COMMAND	DOES
`n8n audit`	Generate a security audit report
`n8n export:workflow --backup --output=…`	Export all workflows
`n8n publish:workflow --id=<ID>`	Publish a workflow
`n8n unpublish:workflow --all`	Unpublish everything (emergency stop)
`n8n execute --id <ID>`	Run a workflow once
`n8n user-management:reset`	Reset users, back to owner setup
`n8n mfa:disable --email=you@example.com`	Turn off 2FA for a locked-out user

CHAPTER 27

What n8n Really Costs

n8n's pricing is simple once you know what's counted. This chapter sets out the plans, the hidden extras (AI and servers), and practical ways to keep your bill low.

IN THIS CHAPTER

- What counts as an execution
- Cloud plans, limits and prices
- Self-hosted costs, including the Business plan
- AI costs: Gateway credits and your own keys
- Ten ways to use fewer executions

27.1 What counts

An **execution** is one full run of a workflow, however many steps it has. Plans count **production executions**: runs started by triggers on published workflows. Manual test runs don't count, and sub-workflow executions don't count towards the monthly limit. For chat agents, each turn of the conversation is one execution.

SCHEDULE	EXECUTIONS PER MONTH (APPROX.)
Once a day	30
Every hour	730
Every 15 minutes	2,900
Every 5 minutes	8,700
Every minute	43,800

27.2 n8n Cloud plans

PLAN	YEARLY BILLING	MONTHLY BILLING	EXECUTIONS
Starter	€20/month	€24/month	2,500
Pro	€50/month	€60/month	10,000
Enterprise	Contact sales	Contact sales	Custom

Pro is also sold with more executions (for example 50,000 a month, with 50 concurrent runs). Prices from n8n.io/pricing, October 2026; yearly billing saves about 17%.

LIMIT	STARTER	PRO	ENTERPRISE
Concurrent executions	5	20 or 50	200+
Max execution duration	5 min	40 min	40 min
Saved executions	2,500	25,000	50,000
Execution log retention	7 days	30 days	Unlimited
Shared projects	1	3	Unlimited
Assistant credits a month	1,600	Up to 9,600	On request
Workflow history	1 day	5 days	365+ days

All plans include unlimited users, unlimited workflows, every integration, AI agents and MCP. Time spent waiting (in a Wait node or for an approval) doesn't count towards the duration limit. Runs over your concurrency limit wait in a queue rather than failing.

27.3 Self-hosted costs

- **Community Edition:** free, unlimited executions. You pay only for the server: a small VPS typically costs a few euros a month, plus a domain name.
- **Business:** €667/month billed yearly (€800 monthly) for 40,000 executions, self-hosted only, adding SSO, environments, Git version control and more shared projects. n8n's Start-up Plan offers 50% off Business for companies with fewer than 20 employees, if you qualify.
- **Enterprise:** contact sales.

Don't forget your time: an hour a month for updates and checks is realistic.

27.4 AI costs

AI model use is billed separately from executions:

- **Gateway credits** (Cloud Starter and Pro): prepaid credit spent per request at the rates on n8n's service pricing page. When the balance hits zero, Gateway-powered nodes stop until you top up; top-ups expire after 12 months.
- **Your own API keys:** the provider bills you directly per token. Set a monthly spending limit in the provider's console.
- **Local models with Ollama** (self-hosted): no per-token cost, but you need capable hardware.

27.5 Ten ways to use fewer executions

- [ ] Use the slowest schedule that does the job
- [ ] Prefer instant (webhook) triggers to frequent polling where possible
- [ ] Filter early, in the trigger's own options if it has them
- [ ] Batch work: one run that handles 100 rows beats 100 runs
- [ ] Move shared steps into sub-workflows (they don't count)
- [ ] Stop chatty triggers, like every Gmail message, with narrower searches
- [ ] Unpublish workflows you no longer need
- [ ] Use Remove Duplicates or data tables to skip already-handled items
- [ ] Pick cheaper AI models for simple jobs like classification
- [ ] Check the Insights dashboard monthly to spot heavy workflows

> **NOTE**
> Prices and limits change. Check n8n.io/pricing before you buy; the figures here were correct in October 2026.

CHAPTER 28

Templates and Community

You rarely need to start from scratch. Thousands of people have already built and shared workflows like yours, and a friendly community is ready to help when you're stuck.

IN THIS CHAPTER

- Using templates safely
- Community nodes
- Getting good help on the forum
- Where to keep learning

28.1 Templates

You rarely need to start from scratch. **n8n.io/workflows** hosts thousands of free templates: lead capture, social media posting, invoice processing, AI support agents, and more. Click **Use workflow** to copy one into your n8n, then add your own credentials.

ADOPT A TEMPLATE SAFELY

1. **Read the description and sticky notes.**
   Good templates explain what to change.
2. **Import it**, then open every node. Replace placeholder IDs, sheet names and channels with yours.
3. **Connect your credentials.**
   Templates never include anyone's secrets.
4. **Test with pinned or sample data** before publishing, especially any node that sends, posts or deletes.

5 **Rename it and add your error workflow.**

28.2 Community nodes

Self-hosted users can also install **community nodes**, extra nodes built by other users, from **Settings › Community nodes**. Install only well-maintained ones from authors you trust. A community node runs with the same access as n8n itself, so treat installing one like installing software on your server. On Cloud, a selection of verified community nodes is available from the nodes panel.

28.3 Getting help

The **community forum** (community.n8n.io) is friendly and active. Search it first; someone has usually solved your problem. When you post, include:

- your n8n version and whether you're on Cloud or self-hosted;
- what you expected and what happened instead;
- the exact error message;
- a small copy of the workflow (select the nodes, copy, and paste them in a code block), with personal data removed;
- a sample of the input data.

PROMPT

Help me write a clear n8n forum question. My setup: [Cloud / self-hosted, version]. Goal: [what the workflow should do]. Problem: [what happens]. Error: [paste]. Make it short, polite and easy to answer.

28.4 Keep learning

- **docs.n8n.io**: the official documentation, including step-by-step learning paths and a glossary.
- **Templates** for the apps you use: study how others built them.
- **The release notes**: a quick weekly skim shows new features.
- **This manual's recipes**: rebuild one with a twist each week.

- [ ] I've imported and adapted one template
- [ ] I check any community node's author and maintenance before installing
- [ ] I know what to include in a forum question

CHAPTER 29

Troubleshooting

When something isn't working, find the closest symptom below. Most problems fall into a handful of patterns, and each has a quick fix.

29.1 Triggers and publishing

SYMPTOM	LIKELY CAUSE	FIX
Schedule never runs	Workflow not published	Publish it; check the button state
Runs at the wrong hour	Time zone	Set it in workflow settings or `GENERIC_TIMEZONE`
Webhook works in testing, not live	Unpublished, or app uses the test URL	Publish; give the app the production URL
Webhook returns 404	Wrong path, method or URL type	Check method (GET/POST) and `/webhook/` vs `/webhook-test/`
Edits have no effect live	Changes still in draft	Publish again
"Published, partial"	A trigger couldn't activate	Hover to see which, fix its credential, republish
Telegram stops after testing	One webhook per bot	Publish again after testing
Polling trigger misses items	Poll interval or filters	Check **Poll Times** and the trigger's filters
Form shows an error after submit	Workflow failed with **Workflow Finishes**	Check Executions for the failed run

29.2 Data and expressions

SYMPTOM	LIKELY CAUSE	FIX
Expression shows `undefined`	Wrong path or capitals	Drag the field in from the input panel
A node runs too many times	It received more items than expected	Check item counts; Aggregate or Limit first
Nodes after a step don't run	That step output zero items	Expected? If not, **Always Output Data**
"Referenced node is unexecuted"	Expression points at a node not on this path	Reference a node that ran, or use Merge
Paired item error	Item links broken	Use `.first()` / `.all()`, or fix the Code node
`[object Object]` in text	Inserted a whole object	Pick a field, or `JSON.stringify()`
Dates off by hours	Time zone or UTC strings	Set workflow time zone; use `.toDateTime()`
Merge outputs nothing	One input received no items	Check both branches ran
File missing in next node	Wrong binary property name	Look at the output's binary slot name

29.3 Credentials and APIs

SYMPTOM	LIKELY CAUSE	FIX
401 Unauthorized	Key wrong or OAuth token expired	Reconnect or replace the credential
403 Forbidden	Missing permission or scope	Grant the scope; reconnect OAuth
429 Too Many Requests	Rate limit	Batching, Loop Over Items + Wait, Retry On Fail
Google "access denied"	App in Testing mode	Add yourself as a test user
Google redirect mismatch	Wrong redirect URI	Paste n8n's exact OAuth Redirect URL
OAuth redirects to `localhost`	Public URL not set (self-hosted)	Set `N8N_EDITOR_BASE_URL`, `N8N_WEBHOOK_URL`
Notion "object not found"	Database not shared with integration	Add the integration to the database
"Credentials could not be decrypted"	Encryption key changed	Restore the original `N8N_ENCRYPTION_KEY`

29.4 AI

SYMPTOM	LIKELY CAUSE	FIX
Agent won't use a tool	Vague tool name or description	Describe exactly when to use it
Agent invents facts	No grounding	Use RAG; "say you don't know"
Forgets earlier messages	No memory or session key changes	Attach memory; check the session key
Agent loops or stops early	Max Iterations	Clarify the task; adjust the limit
Output not in the format needed	Free text	Require Specific Output Format + parser
Knowledge base empty after restart	Simple Vector Store	Use a persistent vector store
Gateway nodes stop working	Credit balance is zero	Top up, or switch to your own key

29.5 Self-hosting

SYMPTOM	LIKELY CAUSE	FIX
Site unreachable	DNS, firewall, or container down	`ping`, `ufw status`, `docker compose ps`
Certificate error	Port 80 closed or DNS wrong	Open 80; check `docker compose logs caddy`
n8n restarting in a loop	Database settings	`docker compose logs n8n`; check `.env`
Webhook URLs show `localhost`	`N8N_WEBHOOK_URL` not set	Set it and restart
Disk full	Execution data	Lower retention; prune; stop saving successes
Out of memory	Big files or huge item lists	Batches, sub-workflows, a bigger server
Lost data after update	Volume removed with `down -v`	Restore from backup; never use `-v` casually
Locked out by 2FA	Lost phone	Recovery codes, or `n8n mfa:disable --email=…`

> **TIP**
>
> Still stuck? Open the failed execution, find the *first* node whose output looks wrong, and ask on the forum with that node's input and error. Chapter 18 has the full routine.

Quick Reference

Pattern: Trigger → get data → reshape → decide → act

Start: n8n Cloud trial (14 days) · or `curl -fsSL https://get.n8n.io | sh` · or Docker `n8nio/n8n` → http://localhost:5678

Editor: N add node · **Execute step** test one node · **Execute workflow** test all · **Publish** (Shift + P) go live · Ctrl/Cmd + K command bar

Expressions: `{{ $json.field }}` · `{{ $('Node name').item.json.field }}` · `{{ $now.toFormat('yyyy-MM-dd') }}` · `{{ $ifEmpty($json.x, 'default') }}`

Code node: `$input.all()` (all items) · `$input.item.json` (each item) · return `[{ json: {...} }]` · Python: `_items`, `item["json"]["field"]`

Webhooks: test URL while listening · production URL only after publishing · secure with header or basic auth

Reliability: Executions tab · Debug in editor · Retry On Fail · error output · error workflow with Error Trigger · pin data while building

AI: AI Agent = chat model + memory + tools + system message · RAG = embeddings + vector store + splitter · MCP Client Tool to use servers · MCP Server Trigger to become one · instance-level MCP for Claude and ChatGPT

Self-hosting: Compose + PostgreSQL + Caddy · `N8N_WEBHOOK_URL`, `N8N_PROXY_HOPS=1`, `N8N_ENCRYPTION_KEY` · back up `.n8n` volume and database · update: edit version, `docker compose pull`, `docker compose up -d`

Glossary

AI Agent node	A workflow node that uses an AI model to decide which attached tools to call to complete a request.
API	A way for programs to talk to each other; n8n's HTTP Request node can call almost any API.
Binary data	File data, such as a PDF or image, carried in an item's `binary` section.
Caddy	A reverse proxy that obtains and renews HTTPS certificates automatically.
Chat model	The sub-node that connects an AI node to a language model provider.
Chunking	Cutting long documents into smaller passages so they can be searched separately.
Cluster node	An AI root node, such as AI Agent, with sub-nodes plugged into it.
Concurrency	How many executions can run at the same time.
Credential	Saved sign-in details, an OAuth connection or an API key, that let nodes use an app.
Cron	A compact text format for schedules, such as `0 9 * * 1-5`.
Data table	A simple table stored inside n8n for keeping data between workflow runs.
DNS record	A setting that points a domain name at a server's IP address.
Docker	Software that runs programs in sealed containers.
Docker Compose	A tool that starts several containers together from one file.
Draft	Saved edits to a workflow that aren't live until you publish.
Embedding	A list of numbers representing the meaning of a piece of text.
Encryption key	The secret n8n uses to encrypt saved credentials.

Environment variable	A setting passed to n8n when it starts, such as `N8N_WEBHOOK_URL`.
Error workflow	A workflow starting with an Error Trigger that runs when another workflow fails.
Execution	One run of a workflow, logged with all its data.
Expression	A formula inside `{{ }}` that inserts or calculates values from earlier nodes.
Gateway credits	Prepaid n8n Cloud credits for running supported AI models without your own provider account.
Guardrails	A node that checks text for unsafe content, personal data or off-topic requests.
Hallucination	When an AI model confidently states something false.
HTTPS	Encrypted web traffic, shown by a padlock in the browser.
Instance	Your own copy of n8n, with its own address.
Item	A single piece of data, such as one row or one email, flowing through a workflow.
Item linking	n8n's record of which input item produced each output item.
JSON	The structured text format n8n uses to represent data.
LLM	Large language model, the AI that reads and writes text.
Luxon	The date library behind `$now` and `$today`.
MCP	Model Context Protocol, the standard for connecting AI apps to tools; see the *MCP* manual.
Memory	A sub-node that stores an AI conversation so the agent remembers it.
Node	One building block in a workflow: a trigger, an app action, or a logic step.
OAuth	A secure "sign in with..." method for connecting apps without sharing your password.
Pin data	Freeze a node's output so you can keep building without re-running it.
Polling	A trigger that checks an app on a timer for new data.
PostgreSQL	A robust database recommended for production n8n.
Production execution	A run started by a trigger on a published workflow; the kind plans count.
Publish	Make the current version of a workflow live so its triggers run automatically.

Queue mode	A scaling setup where workers run executions taken from a Redis queue.
RAG	Retrieval-augmented generation: the AI looks up your documents before answering.
Rate limit	A cap on how many requests a service accepts in a period.
Redis	A fast data store used as the queue in queue mode.
Reverse proxy	A program that receives web traffic, handles HTTPS and passes requests to n8n.
Self-hosting	Running n8n on your own computer or server instead of n8n Cloud.
Sub-workflow	A workflow called by another workflow, used to reuse common steps.
System message	Standing instructions that tell an AI who it is and how to behave.
Task runner	A separate process where n8n runs Code node code safely.
Token	A chunk of text, about three-quarters of a word, that AI providers charge by.
Trigger	The node that starts a workflow, such as a schedule, webhook, form, or app event.
Two-factor authentication	Signing in with a password plus a code from your phone.
Vector store	A database that finds text by similarity of meaning.
VPS	A virtual private server: a rented, always-on computer in a data centre.
Webhook	A unique web address that other apps send data to, which starts a workflow.
Worker	An n8n process that runs executions in queue mode.
Workflow	A set of connected nodes that automates a task.
Workflow history	Saved earlier versions of a workflow that you can view or restore.